A Federated and Incremental Network Intrusion Detection System for IoT Emerging Threats

Raffaele Carillo, Francesco Cerasuolo, Giampaolo Bovenzi,
Domenico Ciuonzo, *Senior Member, IEEE*, Antonio Pescapé, *Senior Member, IEEE*

***Abstract*—Ensuring network security is increasingly challenging, especially in the Internet of Things (IoT) domain, where threats are diverse, rapidly evolving, and often device-specific. Hence, Network Intrusion Detection Systems (NIDSs) require (i) being trained on network traffic gathered in different collection points to cover the attack traffic heterogeneity, (ii) continuously learning emerging threats (viz., 0-day attacks), and (iii) be able to take attack countermeasures as soon as possible.**
In this work, we aim to improve Artificial Intelligence (AI)-based NIDS design & maintenance by integrating Federated Learning (FL) and Class Incremental Learning (CIL). Specifically, we devise a Federated Class Incremental Learning (FCIL) framework–suited for early-detection settings—that supports decentralized and continual model updates, investigating the non-trivial intersection of FL algorithms with state-of-the-art CIL techniques to enable scalable, privacy-preserving training in highly non-IID environments. We evaluate FCIL on three IoT datasets across different client scenarios to assess its ability to learn new threats and retain prior knowledge. The experiments assess potential key challenges in generalization and few-sample training, and compare NIDS performance to monolithic and centralized baselines.

***Index Terms*—Network Intrusion Detection Systems, Internet of Things, Federated Learning, Class Incremental Learning, 0-day attacks**

I. Introduction

CYBER-ATTACKS have increased rapidly in the last few years, reaching an impressive number of 600 million of attacks performed daily [1], highlighting the urgent need for service providers to adopt a more proactive role in ensuring the integrity, confidentiality, and availability of data across digital infrastructures. In this scenario, NIDSs are central to securing modern networks, shifting from static, signature-based methods to adaptive approaches powered by Machine Learning (ML) and Deep Learning (DL) techniques [2]. This need is even more pronounced in the IoT domain, where resource-constrained and heterogeneous devices increase the attack surface. For instance, IoT nodes are often hijacked into botnets for large-scale DDoS attacks (e.g., Mirai) [3], or run outdated firmware that enables persistent exploitation [4].

However, AI-based NIDS cannot easily adapt to emerging (0-day) attacks, as updating them requires *training-from-scratch*, which is both storage- and computation-intensive [5]. Moreover, while cross-network data sharing could enhance NIDS by broadening threat coverage, it raises serious *privacy concerns*, making it *impractical in most real-world scenarios* involving user data. Notably, recent studies have investigated either CIL or FL for NIDS, but largely in isolation: CIL approaches typically assume centralized access to all traffic data, while FL ones usually consider a fixed and pre-defined set of attack classes. Only a limited number of works explore their integration, and those that do often focus on simplified scenarios such as binary detection tasks, overlapping classes, or post-hoc analysis. Consequently, the problem of incrementally learning *new, client-specific attack classes* from *decentralized, non-IID data* in an *early-detection* setting remains largely unaddressed.

In this work, we propose a FCIL framework for NIDS that simultaneously addresses the challenges of incremental learning and decentralized training. By integrating CIL, the system can continuously incorporate new attack classes without forgetting previously learned threats, while FL enables collaborative model training across distributed clients, ensuring scalability and preserving data privacy. This combination allows our NIDS to adapt in real time to emerging threats in heterogeneous network environments [6–8].

Combining FL and CIL for designing NIDS poses *unique and still largely unaddressed challenges*. First, the non-IID nature of client data means new attack classes may emerge only on specific clients, leading to imbalanced updates and poor generalization. Second, catastrophic forgetting is intensified, as clients may observe only new classes, causing the global model to overwrite knowledge of previously learned attacks. Third, the stability–plasticity dilemma becomes even more pronounced, as each client must not only retain previously learned classes while learning new (local) ones, but also integrate new disjoint attack classes from other clients.

Hence, the *contributions* of this work are the following:

- we formalize and study a realistic FCIL problem for NIDS, where *early detection*, *client-specific novel attacks*, and *non-IID decentralized data* jointly coexist, a setting that has not been systematically addressed in prior work;
- we provide a comprehensive analysis of how state-of-the-art (SOTA) CIL strategies behave in federated environments, revealing non-trivial interactions with different aggregation schemes (`FedAvg`, `FedProx`, and `FedDyn`) and limitations, absent in centralized setups;
- we investigate realistic federated configurations with varying numbers of clients (i.e., 2 and 5), each introducing a *single, disjoint novel attack*, thereby explicitly

R. Carillo, F. Cerasuolo, G. Bovenzi, D. Ciuonzo, and A. Pescapé are with the Department of Electrical Engineering and Information Technologies (DIETI) at the University of Naples Federico II, Italy (name.surname@unina.it). The illustrations in this manuscript include images downloaded from *flaticon.com* and *fontawesome.com*.

Table I: Studies proposing CIL and/or FL techniques for NIDS from 2022 onwards in chronological order.

Paper	Early NIDS	DL	NIDS Task	CIL			FL				Few Shot	IoT Threats	TO	Input Data	ML/DL Model
				Setup	Approach	Increments	Setup	Approach	#Clients	DC					
[9] (2022)	✗	✗	mMD	✓	Hard Sampling	MCA	✗	-	-	-	✗	○	B	Statistics	AB, HT
[10] (2023)	✗	✓	mMD	✓	GDUMB, iCaRL, GSS, ER, ASER, MIR, AGEM, SSR, NCM, SLDA, EWC, LwF	MCA	✗	-	-	-	✗	○	F	Statistics	1D-CNN
[11] (2023)	✓	✓	mMD	✓	I^2RNN	SCA MCA	✗	-	-	-	✗	○	S	PL, IAT, DIR, WIN, TTL, FLG	LSTM
[12] (2023)	✓	✓	mMD	✓	FT, iCaRL, BFS-NID FOSTER, IDFSCIL, LIMIT, C-FSCIL	MCA	✗	-	-	-	✓	○	B	PB*	Transformer
[13] (2024)	✓	✗	mMD	✓	FT, EEIL, LwF *	MCA	✗	-	-	-	✗	○	B	PL, IAT, DIR, WIN, TTL, FLG	MLP
[14] (2024)	✗	✓	mMD	✓	FT, EWC, LwF, iCaRL, SimpleCIL, SPCIL	MCA	✗	-	-	-	✗	○	B	Statistics*	ResNet18
[15] (2025)	✓	✓	bMD AC	✓	FT, FT-Mem, LwF, iCaRL+, BiC, EWC EEIL, LUCIR	SCA	✗	-	-	-	✓	●	B	PL, IAT, DIR, WIN,	2D-CNN
[16] (2025)	✓	✓	bMD mMD	✓	FT, FT-Mem, BiC	MCA	✗	-	-	-	✗	●	B	PL, IAT, DIR, WIN,	2D-CNN
[17] (2022)	✗	✗	bMD AD	✗	-	-	✓	FedAvg†	8	✗	✗	◐	S	Statistics	MLP AE
[18] (2022)	✗	✓	mMD	✗	-	-	✓	FedAvg	2	✗	✗	○	B	Statistics	GRU
[19] (2022)	✗	✓	mMD	✗	-	-	✓	FedAvg	5 10 15	✗	✗	◐	B	Statistics	DNN CNN RNN
[20] (2023)	✗	✓	mMD	✗	-	-	✓	DAFL	2 5 7	✗	✗	○	B	Statistics	CNN
[21] (2023)	✗	✗	AD	✗	-	-	✓	FedAvg FedProx	10	✗	✗	○	B	Statistics	AAE
[22] (2024)	✗	✓	mMD	✗	-	-	✓	FedAvg	2 4	✗	✗	○	B	Statistics	CNN
[23] (2022)	✓	✓	bMD	✓	SOINN-RBF	MCA	✓	FedAvg, CMFL, Fed-SOINN	10	✗	✗	○	B	IAT, IDL, ACT, BR	2D-CNN
[24] (2023)	✗	✗	bMD	✓	FT	MCA	✓	FedAvg	10	✗	✗	○	F	Statistics	MLP
[25] (2023)	✗	✓	mMD	✓	iCaRL	MCA	✓	FedAvg†	3	✗	✗	●	F	Statistics	CNN
[26] (2024)	✓	✓	mMD	✓	iCaRL+	MCA	✓	GLFC	10	✗	✗	○	B	PS, PL, IAT, DIR, WIN, FLG, ACT, IDL	2D-CNN
[27] (2024)	✗	✓	mMD	✓	FT-Mem, iCaRL, WA, DER	MCA	✓	FedAvg†	3	✗	✗	●	F	Statistics	1D-CNN
[28] (2024)	✓	✓	bMD	✓	iCaRL+	SCA	✓	EvoFedIDS, FedSI FedSpace, GLFC FCIL, FedProto	5	✗	✗	○	B	PS, PL, IAT, DIR, WIN, FLG, ACT, IDL	2D-CNN
[29] (2024)	✗	✓	mMD	✓	EWC	MCA	✓	FedAvg†	50	✗	✗	◐	S	Statistics	CNN-GRU
[30] (2025)	✗	✓	mMD	✓	GEM, EWC iCaRL	MCA	✓	FedAvg, FedProx, SCAFFOLD, FedMD, FedDF, FedGKT	10	✗	✗	○	F	Statistics	BERT
This work (2025)	✓	✓	bMD mMD AC	✓	iCaRL+, BiC, BiC$^+$, MEMENTO, MEMENTO$^+$	SCA	✓	FedAvg FedProx FedDyn	2 5	✓	✓	●	B	PL, IAT, DIR, WIN, TTL, FLG	2D-CNN

Deep Learning (DL); **Task**: binary Misuse Detection (bMD), multi-class Misuse Detection (mMD), Anomaly Detection (AD), Attack Classification (AC); **CIL Increments**: Single Class Addition (SCA), Multi-Class Addition (MCA); **Disjoint Classes (DC)**; **Traffic Object (TO)**: Biflow (B), Session (S), Flow (F); **Input Data**: Packet Length (PL), Inter-Arrival Time (IAT), Direction (DIR), TCP Windows Size (WIN), TCP Flags (FLG), Time-To-Live (TTL), Payload Bytes (PB), Idle Time (IDL), Active Time (ACT), Byte Rate (BR), L4 Payload Size (PS), Contextual Features (CF); **Machine Learning (ML)/DL Model**: Convolutional Neural Network (CNN), Long Short Term Memory (LSTM), Multi Layer Perceptron (MLP), Gated Recurrent Unit (GRU), Auto Encoder (AE), Hoeffding Tree (HF), AdaBoost (AB); *: image-like format; †: with modifications; ✓ present, ✗ lacking; ●: more than one IoT dataset; ◐: a single IoT dataset; ○: no IoT dataset.

stressing FCIL-designed NIDS under severe class imbalance and client heterogeneity;
- we evaluate FCIL-designed NIDS in *real-world scenarios*, assessing its ability to (*i*) generalize across networks and (*ii*) learn effectively from a limited number of samples.

The rest of the paper is organized as follows. The studies dealing with CIL and FL for NIDS are reviewed in Sec. II. Then, Sec. III describes the FCIL procedure. The experimental setup and results are reported in Sec. IV and V, respectively. Lastly, Sec. VI concludes our work, highlighting also future directions of our research.

II. Related Works

In this section, we review the use of CIL and FL in the context of NIDS. First, we discuss approaches that apply CIL to NIDS in Sec. II-A, followed by an overview of FL-based methods in Sec. II-B. Then, we examine recent efforts that integrate both paradigms—CIL and FL—for NIDS (Sec. II-C). Finally, Sec. II-D positions our work within this landscape. A summary of the reviewed literature is provided in Tab. II.

A. Incremental Learning for NIDS

In this section, we review the literature on CIL applied to NIDS, focusing on approaches that adapt existing intrusion detectors to counter emerging attack types and anomalies. In the literature, CIL approaches are divided into *three main families* based on model expansions [31]: (*i*) *fine-tuning*, expanding the model head and updating the entire model with the incremental training set; (*ii*) *fixed-representation*, expanding the model head and freezing parts of the pre–trained model (viz., backbone and/or old head), definitively training the non-frozen parts; (*iii*) *model-growth*, adding new models or layers to accommodate new knowledge. All the reviewed works are from the fine-tuning family, except for `I`^{2}`RNN` [11] and `SPCIL` [14], which employ model-growth solutions. Notably, the fixed-representation family remains largely unexplored for network intrusion detection tasks, as freezing the base model may undermine the plasticity required to learn new patterns (see column "**CIL Approach**" in Tab. I for further details).

As for incremental episodes (column "**CIL Increments**"), most of the reviewed works adopt multi-class increments—i.e., adding more new classes per episode—while only [11, 15] explore the single-class addition scenario, which more closely mirrors real-world deployment with new attacks typically appearing individually over time.

Reviewed work deals with the Multi-class Misuse Detection (mMD) task—i.e., distinguishing among benign and all the attack classes—, while Binary Misuse Detection (bMD)—i.e., differentiating benign and malicious traffic—is also considered in [16]. In addition, Cerasuolo et al. [15] decompose the intrusion detection process into two sequential tasks: a bMD task and an Attack Classification (AC) task—i.e., categorization of malicious traffic into different attack types.

Crucially, most of these studies focus on *post-mortem detection*—that is, they do not perform *early detection* (see column "**Early Detection**")—typically relying on statistics computed over the entire traffic object (see column "**Input Data**"). In contrast, a subset of works avoid (handcrafted) feature summarization, using raw fields from packet headers or payloads as direct inputs to the model. These approaches typically leverage only the first N_p packets of a flow to build NIDS that support *early-stage decisions* and enable *timely, proactive response* to threats.

B. Federated Learning for NIDS

In this section, we review the literature on FL applied to NIDS. FL allows NIDS to be trained on decentralized attack data, preserving privacy and minimizing malicious traffic transmission. This is especially valuable for sensitive or large-scale data, enabling collaborative and privacy-preserving training [8]. In the literature, *two main FL families* are commonly distinguished based on how data is distributed among clients: (i) *horizontal (or homogeneous) FL*, where clients share the same feature space but each possesses a different set of data samples; and (ii) *vertical (or heterogeneous) FL*, in which clients hold different features corresponding to the same sample set [32]. Additionally, FL can be classified by the *network dimensions* into (i) *cross-device* and (ii) *cross-silo* settings. The former involves many personal devices with unstable connections and variable data, while the latter features fewer entities—e.g., organizations—with more stable, high-bandwidth links and less frequent communication [33]. Furthermore, client data can be characterized as either *independently and identically distributed* (IID) or *non-IID*. In the former setting, data across clients is statistically consistent and independent. Conversely, in the latter case, data samples are interdependent and drawn from different distributions, often reflecting the inherent heterogeneity among clients [33].

Among the reviewed literature, all works deal with *horizontal* FL, sharing feature space while having a disjoint sample space. Moreover, all the works implement a *cross-device* setup, with local training performed on edge devices, and deal with classes and samples shared among clients, resulting in data distributions that are only partially non-IID (column "**DC**").

As for FL algorithm (column "**FL approach**"), the majority of works leverage `FedAvg` [17–19, 21, 22]. Additionally, other works propose minor modifications to `FedAvg`, such as incorporating weighted averaging schemes and client selection based on contribution significance [20], while others adopt more sophisticated FL algorithms, such as FedProx [21]. Interestingly, Rey et al. [17] investigate techniques to *defend* against adversarial clients aiming to poison the global model.

Finally, all the revised studies address the mMD task employing (supervised) DL models [18–20, 22], except for [17] and [21] leveraging the (unsupervised) AutoEncoders (AEs) to perform the Anomaly Detection (AD) task—that is, training only on the benign class and recognizing attacks as anomalies or out-of-distribution samples.

C. Federated Incremental Learning for NIDS

In this section, we review existing work that integrates incremental expansion of NIDS (i.e., CIL) with decentralized training approaches (i.e., FL), namely, FCIL. Notably, all these studies focus on a multi-class addition (column "**CIL Setup**")—only [28] addresses single-class addition—and consider non-disjoint classes among clients (column "**DC**").

Among these studies, most adopt CIL approaches which are baselines in network-related tasks, except for `SOINN-RBF` [23]— leveraging self-organizing incremental neural networks (`SOINN`) in conjunction with Radial Basis Function (`RBF`), incrementally mapping the original input into a higher-dimensional space.

Regarding the FL strategy, most employ `FedAvg` either in its original form or with minor adaptations, as seen in [25, 27, 29]. Notably, Jin et al. [25] leverage class prototypes and importance factors for client models in the aggregation process. On the other hand, Mao et al. [27] and Quyen et al. [29] weight each client model during `FedAvg` aggregation according to the size of its training dataset. In addition, several studies adopt alternative FL aggregation algorithms as performance baselines for comparison, including FedSI [28], CMFL [23], FedSpace [28], GLFC [26, 28], FCIL [28], FedProto [28], FedProx [30], SCAFFOLD [30], FedMD [30], FedDF [30], and FedGKT [30], or define new approaches, such as EvoFedIDS [28] or Fed-SOINN [23].

As for the detection task, the majority focus on supervised misuse detection—either bMD [23, 24, 28] or mMD [25, 26, 29]. Notably, Jin et al. [25] detect both known and zero-day intrusions and continuously adapt to emerging threats, but also perform mMD to discriminate among known classes.

D. Literature Gap & Positioning

In this work, we deal with *incremental* and *decentralized* NIDS to tackle the evolving nature of cyber-attacks and associated privacy concerns. Differently from previous works considering only CIL [9–16] or FL [18–22] separately, which either assume centralized data availability or a fixed and closed set of attack classes, we explore how NIDS *can continuously learn in a decentralized fashion*.

Equally important, in contrast with existing literature on FCIL for NIDS [23–25, 27–29], which mostly assumes full-traffic or post-mortem analysis and overlooks the constraints of timely intrusion response, we consider an *early-detection* setup and investigate the mMD task, along with its decoupled counterpart bMD, followed by the AC task. Separating bMD and AC decouples detection from classification performance. Unlike mMD, this distinction reveals whether a system can only detect threats and correctly identify them separately, *enabling a clearer assessment of its capabilities*.

Furthermore, in order to support low-resource environments, we adopt neither model-growth approaches (e.g. `DER`)—given their increasing storage and communication costs, which are particularly detrimental in federated settings—, nor baseline approaches for network-related tasks (i.e., `FT`, `EWC`, `iCaRL`, and `WA`), which have been shown to be non-effective for network traffic data [34]. Conversely, *we focus on cutting-edge CIL approaches from fine-tuning family*, such as `iCaRL+` [35], `BiC` [36] and `BiC`$^+$ [37], `MEMENTO` [38] and `MEMENTO`$^+$ [37], in their *decentralized flavor*. This choice is motivated by the lightweight nature of fine-tuning approaches, which impose minimal storage and communication overhead when compared to model-growth solutions.

To account for non-IID and unbalanced attack distributions, we move beyond the *de facto* standard `FedAvg` and include `FedProx` and `FedDyn`, which explicitly handle client heterogeneity and update stability, enabling *a more realistic evaluation of CIL–FL integration*.

Further, most prior FCIL studies assume overlapping attack classes or the simultaneous introduction of multiple classes, which mitigates class imbalance and simplifies aggregation. In contrast, we consider a *more general scenario* in which each client introduces a *single, distinct novel attack*, resulting in client-specific attack distributions—i.e., no class overlap between clients. Moreover, we consider the addition of a *single attack (class) per client*—a more challenging scenario than those considered in prior work, due to the pronounced imbalance between new and existing classes [34]. Additionally, adding a single attack per client reflects realistic scenarios where new attacks emerge sparsely and unevenly across the network. Lastly, we assess scalability by varying the number of clients in federated networks, evaluating how the resulting FCIL NIDS handles real-world challenges like cross-network generalization and limited data availability.

III. Methodology

We begin by presenting the core methodologies behind CIL (Sec. III-A) and FL (Sec. III-B). Then, we introduce the NIDS-focused unified framework for FCIL (Sec. III-C), which integrates the key principles of both paradigms.

A. Class Incremental Learning

In this work, we adopt the CIL paradigm within DL architectures. To provide context, we first introduce the notation about data and fundamental concepts underlying both approaches. Notably, the training dataset ($\mathcal{D} = \{(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_N, y_N)\}$) is composed of N input samples, where each sample (e.g., a biflow) $\boldsymbol{x}_n \in \mathcal{X}$ is labeled as benign traffic or attack class (i.e., $y_n \in \mathcal{K} = 1, 2, 3, \ldots$).

Considering a generic DL architecture, it is composed of two parts: a *backbone*, in charge of the feature extraction $\phi : \mathcal{X} \rightarrow \mathcal{V}$ obtaining from the input ($\boldsymbol{x}$) its latent space representation $\boldsymbol{v}$ (viz., feature vector); a *head*, transforming feature vectors into logits $h(\phi(\boldsymbol{x})) = \boldsymbol{o}(\boldsymbol{x}) \in \mathcal{O}$. Then, logits are converted into class probabilities via the usual softmax activation function, defined as

$$[\mathit{softmax}(\boldsymbol{x})]_k = p_k(\boldsymbol{x}; \mathcal{K}) = e^{o_k(\boldsymbol{x})} / \sum_{j=1}^{|\mathcal{K}|} e^{o_j(\boldsymbol{x})}, \qquad (1)$$

and predictions are obtained as $\hat{y} \triangleq \operatorname{argmax}_{k \in \mathcal{K}} \{p_k(\boldsymbol{x}; \mathcal{K})\}$.

The incremental NIDS aims to train a set of parameters $\boldsymbol{\theta}$ to learn a function $\gamma : \mathcal{X} \rightarrow \mathcal{K}$, with $\gamma(\boldsymbol{x}) = \hat{y} \in \mathcal{K}$ representing the predicted label for input x (i.e., legitimate traffic or a specific attack type).

The CIL problem can be formulated as follows: at each episode, a set of new classes $\mathcal{K}^{new}$ is introduced, along with a labeled dataset $\mathcal{D}^{new}$. The goal is to update an existing NIDS—previously trained on $\mathcal{D}^{old}$—to integrate knowledge from $\mathcal{K}^{new}$ while preserving its ability to recognize the previously learned classes $\mathcal{K}^{old}$, thereby enabling discrimination over the complete class set ($\mathcal{K}^{all} = \mathcal{K}^{old} \cup \mathcal{K}^{new}$). Some CIL methods additionally exploit a subset of samples from the old classes, retained in memory, to reinforce prior knowledge during incremental updates. In such cases, training is performed on the combined dataset $\mathcal{D}^{mem} \cup \mathcal{D}^{new}$ [39].

In the following, the CIL approaches employed in this work are presented. Specifically, we consider approaches from *fine-tuning* family, as they offer the highest scalability without increasing the resource demand. [31]. Notably, all these approaches are considered as a baseline for the incremental process—i.e., `FT-Mem`—or have proven to be effective for network traffic classification [34, 35, 38] or adaptive NIDS task [15]. They utilize an exemplar memory and often employ a composite loss function to guide the training process, with an optional bias correction layer to prevent *task-recency bias*.

`FT-Mem`**—Fine-Tuning with Memory** [34]: built on top of Fine-Tuning (`FT`) approach [40], it stores a small part of old classes samples—selected with herding strategy—and use them in the incremental training (viz., $\mathcal{D} = \mathcal{D}^{new} \cup \mathcal{D}^{mem}$) to reduce catastrophic forgetting, by refreshing old knowledge.

`iCaRL+`**—Incremental Classifier and Representation Learning Plus** [35]: it adapts `iCaRL` to network traffic, incorporating both rehearsal strategy (through herding selection) and a composite loss function with equally weighted classification and Knowledge Distillation (KD) terms ($\lambda_{class} = \lambda_{dist} = 1$). Differently from `iCaRL` employing a Nearest Mean Classifier, `iCaRL+` leverage a model head that dynamically expands and activates through the softmax function.

`BiC`**—Bias Correction** [36]: it introduces an *additional layer*—parametrized with α and β—to correct new classes logits to reduce the task recency and sample biases towards new classes. The incremental training is led by a composite loss function with classification and KD terms (weighted by $\lambda_{class} = |\mathcal{K}^{new}|/|\mathcal{K}^{all}|$ and $\lambda_{dist} = 1 - \lambda_{class}$, respectively). `BiC` retains a balanced small set of biflows—also called *correction set*—composed of $\lceil \mathcal{P}\% \cdot N_{old} \rceil$ for each old and new class, where N_{old} represents the number of old samples for each $\mathcal{K}^{old}$.[1] Therefore, the additional layer is trained on this dataset (viz., correction set) to determine the two parameters with both backbone and head frozen.

`MEMENTO`**—distill augMented knowlEdge, sMooth nEw iNformation, and recTify bOth** [38]: built upon `BiC`, it is defined depending on the scenario considered. Notably, for few-class increments, `MEMENTO` introduces a series of enhancements for improving network traffic classification performance. These include (i) biflow augmentation techniques using static traffic primitives applied to the IAT—viz., jittering

$$\mathbf{IAT}_{jit} \triangleq \begin{bmatrix} \mathrm{iat}_1 & \cdots & \mathrm{iat}_{N_p} \end{bmatrix} + \begin{bmatrix} \rho_1 & \cdots & \rho_{N_p} \end{bmatrix},$$

where $\rho_i \sim \mathcal{N}(\mu, \sigma)$, $i = 1, \ldots, N_P$, and translating

$$\mathbf{IAT}_{tra} = \begin{bmatrix} \mathrm{iat}_1 & \cdots & iat_{N_p} \end{bmatrix} + \begin{bmatrix} \rho & \cdots & \rho \end{bmatrix},$$

where $\rho \sim \mathcal{N}(\mu, \sigma)$—, ($ii$) the construction of a balanced validation set from exemplars memory to lead incremental training across tasks, and (iii) the use of a higher temperature in the classification loss to encourage softer probability distributions and reduce overconfidence. Last but not least, `MEMENTO` applies (iv) output smoothing during knowledge distillation to better handle biflows belonging to newly-introduced classes.

B. Federated Learning

In a typical FL setup, a network of M clients $c_1, c_2, \ldots, c_M$ collaboratively trains a shared model. During each communication round r, mth client first updates its local model $\boldsymbol{\theta}_m^r$ using its private dataset $\mathcal{D}_m$, tailoring the training to its task. Once local training is complete, clients send their updated model parameters to a central federated server, which coordinates the training process. The server aggregates the received models to produce an updated global model that captures the collective knowledge of all clients. This global model is then redistributed to the clients, who resume local training based on the aggregated update. This process is iteratively repeated for $\mathcal{R}$ communication rounds.

[1]The number of per-class biflows in the correction set is limited by the old classes, which are fewer in memory than the new-class biflows of the current episode.

In this work, we consider a *cross-silo* FL setting, i.e., a federated network with a small number of clients representing different organizations or geographical collection points. Each client corresponds to a group of IoT devices (e.g., gateways, edge nodes, or administrative domains) rather than individual constrained sensors, and clients collaboratively train a global NIDS; while each client observes traffic from different attack classes (*non-IID*), all employ the same feature space (*horizontal* FL). Communication is therefore limited to exchanging model updates, whose size—on the order of a few MB per round for our architecture—is compatible with NB-IoT, LTE-M, and typical edge/cloud backhaul links assumed in cross-silo IoT and industrial deployments [41]. Supporting fully constrained end devices would instead require additional mechanisms such as model compression or hierarchical FL.

In the experiments, we leverage different FL approaches to aggregate models from different federated clients. In detail, we employ three approaches, described in the following. Notably, we exclude some approaches from our analysis as they compute and share local prototypes across the federation, introducing significant computational and communication overhead—e.g., FedProK [42], LGA [43].

Federated Averaging (FedAvg) [44]: each client performs local model update by minimizing its own loss function $F_m(\boldsymbol{\theta})$, yielding updated parameters $\boldsymbol{\theta}_m^r = \arg\min_{\boldsymbol{\theta}} F_m(\boldsymbol{\theta})$. These locally-updated models are then transmitted to a central server, which computes the global model by averaging the collected client parameters.

$$\boldsymbol{\theta}^r = \frac{1}{M} \sum_{m=1}^{M} \boldsymbol{\theta}_m^r \tag{2}$$

FedProx [45]: built upon `FedAvg`, it is based on the assumption that clients may have very different data distributions and may not converge when using `FedAvg`. For these reasons, `FedProx` adds a *proximal term* to the local (viz. client) objective function to keep local updates closer to the global model. At each communication round r, client m computes updated parameters as $\boldsymbol{\theta}_m^r = \arg\min_{\boldsymbol{\theta}} L_m(\boldsymbol{\theta})$ by minimizing the following augmented objective function:

$$L_m(\boldsymbol{\theta}) = F_m(\boldsymbol{\theta}) + \frac{\mu}{2} \left\| \boldsymbol{\theta} - \boldsymbol{\theta}^{r-1} \right\|^2, \tag{3}$$

where $F_m(\cdot)$ is the standard local loss on client m, and μ controls the strength of the regularization term. The squared ℓ_2-norm keeps each client's model close to the previous global parameters $\boldsymbol{\theta}^{r-1}$, enhancing robustness to data heterogeneity. Server aggregation is then performed analogously as `FedAvg`.

FedDyn [46]: it aims to improve the communication efficiency of `FedAvg` by dynamically modifying the client objective with a penalty term so that when model parameters converge, they do so in stationary points of the global empirical loss. In particular, the linear and quadratic penalty terms minima are consistent with the global loss stationary point. The penalty term is based on the local and server models, namely:

$$L_m(\boldsymbol{\theta}) = F_m(\boldsymbol{\theta}) - \langle \nabla F_m(\boldsymbol{\theta}_m^{r-1}), \boldsymbol{\theta} \rangle + \frac{\alpha}{2} \left\| \boldsymbol{\theta} - \boldsymbol{\theta}^{r-1} \right\|^2, \tag{4}$$

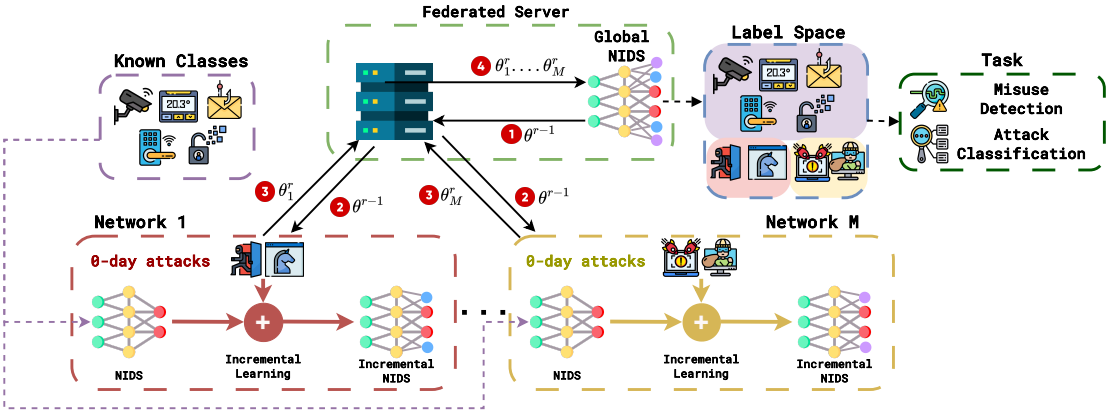


Figure 1: Visual representation of the overall proposal. Each client incrementally trains its local NIDS, including new attacks, and simultaneously participates in the federated network's collaborative information exchange. This process converges to a global NIDS, then distributed across all clients, to perform (*binary* and *multiclass*) misuse detection and attack classification.

where the first term represents the standard local empirical loss (e.g., cross-entropy); the second term ($\langle \nabla F_m(\boldsymbol{\theta}_m^{r-1}), \boldsymbol{\theta} \rangle$) subtracts a linear approximation based on past gradients to compensate for bias in the client distribution; and the third term, $\frac{\alpha}{2} \left\| \boldsymbol{\theta} - \boldsymbol{\theta}^{r-1} \right\|^2$, encourages the local model to stay close to the previous global model $\boldsymbol{\theta}^{r-1}$, similar to `FedProx`. Server aggregation is performed similarly to `FedAvg` and `FedProx`.

C. Federated Class Incremental Learning

In the FCIL setup, each client incrementally introduces a new set of classes, with the objective of training a global NIDS capable of accurately classifying both previously learned and newly introduced classes across all clients. The overall procedure is depicted in Fig. 1.

Formally, at the generic (incremental) episode, mth client in the federated network adds a set of new classes $\mathcal{K}_m^{new}$. The FCIL procedure begins with a base model—trained on previously seen classes $\mathcal{K}^{old}$ during an earlier federated (possibly incremental) iteration—which is then distributed to all clients joining the network. Accordingly, all the clients start the incremental process from the same model, effectively representing a scenario where clients can join the network asynchronously and require a synchronization step. In a federated setting, m client performs local training for a fixed number of epochs before synchronizing with the server and other clients across $\mathcal{R}$ communication rounds.

In the FCIL context, the goal is to update an existing NIDS—previously trained on $\mathcal{D}^{old}$—to integrate knowledge from $\bar{\mathcal{K}}^{new} \triangleq \bigcup \{\mathcal{K}_m^{new}\}_{m=1}^{M}$ (i.e. the set of new classes from *all* the involved clients) while preserving its ability to recognize the previously learned classes $\mathcal{K}^{old}$. Hence, discrimination capabilities of the updated NIDS are required to be over the complete class set $\mathcal{K}^{all} = \mathcal{K}^{old} \cup \bar{\mathcal{K}}^{new}$.

Despite the generality of the above FCIL formulation (and of the proposed solutions), in this work we are concerned with the case where *each client introduces a distinct set of new classes*, i.e. $\mathcal{K}_m^{new} \cap \mathcal{K}_{m'}^{new} = \emptyset$, for $m \neq m'$. This setting poses a significant challenge, as it leads to an extreme form of non-independent and identically distributed (non-IID) data distribution among the clients, who must also mitigate catastrophic forgetting as they learn new classes.

On the top of that, integrating CIL into a federated setting introduces specific design challenges. While simpler *fine-tuning* methods can be directly federated, those including post-training techniques like bias correction [36, 38] require tailored solutions. In this work, we leverage *two different solutions* [37] which capitalize FL framework.

In the first one, all clients share the same bias layer for new classes $\bar{\mathcal{K}}^{new}$, similar to `BiC`-based solutions in a centralized setting. Each client updates this layer using its own data ($\mathcal{D}^{mem} \cup \mathcal{D}_m^{new}$), the server averages the clients' parameters $(\alpha_m^{new}, \beta_m)$ to compute the global correction, and sent the averaged correction to all clients for the next round:

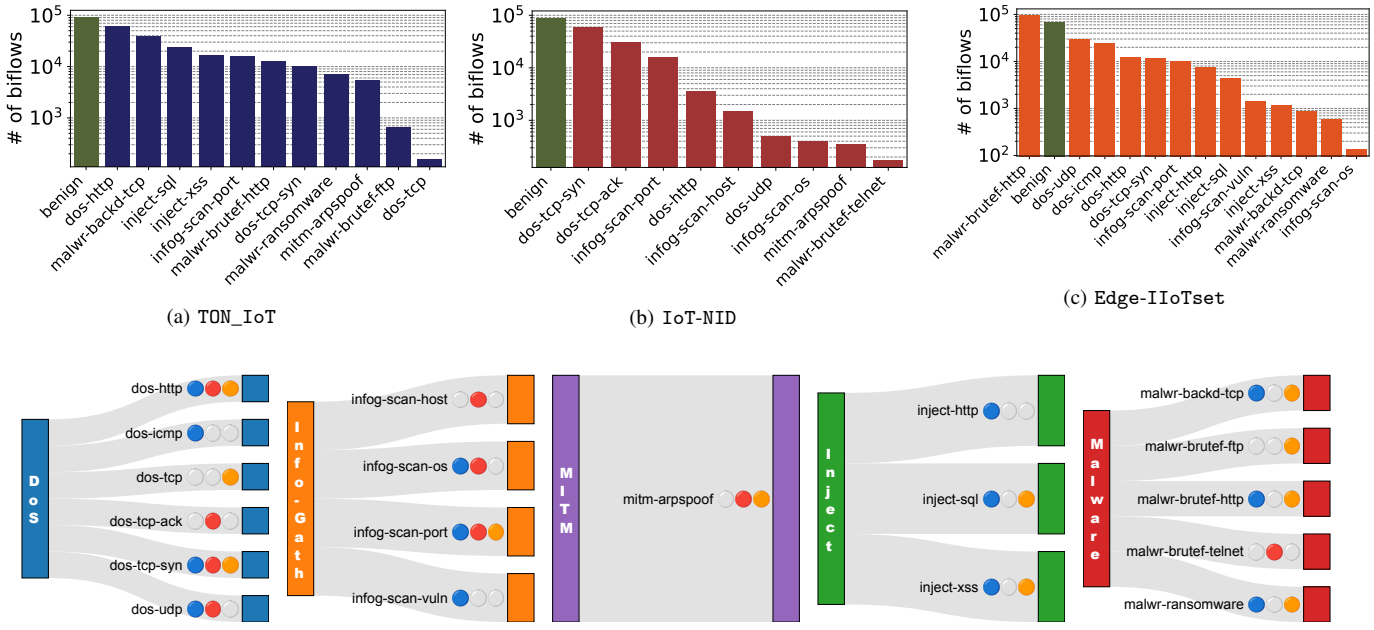
$$\bar{\boldsymbol{o}}(\boldsymbol{x}) = \begin{bmatrix} \boldsymbol{o}^{old}(\boldsymbol{x}) \\ \frac{1}{M} \sum_{m=1}^{M} \left(\alpha_m^{new} \boldsymbol{o}^{new}(\boldsymbol{x}) + \beta_m \cdot \mathbf{1}_{|\bar{\mathcal{K}}^{new}|} \right) \end{bmatrix} \tag{5}$$

In the second approach, each client maintains a personalized bias layer for its new classes. At each round, client c_m updates only its local parameters $(\alpha_m^{new}, \beta_m)$ and shares them with the server. The server stacks all client-specific corrections to support classification across all new classes:

$$\bar{\boldsymbol{o}}(\boldsymbol{x}) = \begin{bmatrix} \boldsymbol{o}^{old}(\boldsymbol{x}) \\ \alpha_1^{new} \boldsymbol{o}_1^{new} + \beta_1 \cdot \mathbf{1}_{|\mathcal{K}_1^{new}|} \\ \vdots \\ \alpha_M^{new} \boldsymbol{o}_M^{new} + \beta_M \cdot \mathbf{1}_{|\mathcal{K}_M^{new}|} \end{bmatrix} \tag{6}$$

The set of all correction parameters $(\alpha_m^{new}, \beta_m)_{m=1}^{M}$ are sent back to all clients for next round. We denote the averaged variant by $\mathcal{A}$, and the client-specific one by $\mathcal{A}^+$.

We note that the proposed FCIL framework does not incur additional asymptotic computational overhead compared to standard FL. The overall complexity is dominated by local training at each client and can be expressed as $\mathcal{O}(\sum_{m=1}^{M} R\, E\, N_m\, P)$, where M is the number of clients, R the number of communication rounds, E the number of local training epochs, N_m the number of training samples at client



(a) TON_IoT (b) IoT-NID (c) Edge-IIoTset

Figure 2: Biflows distributions for the IoT datasets employed. Green bars represent the benign class across all three datasets. Sankey diagrams show specific attacks per category, with colored circles indicating the datasets that contain each attack.

m (i.e., the size of $\mathcal{D}^{mem} \cup \mathcal{D}_m^{new}$), and P the number of trainable model parameters. The server-side aggregation of client parameters has linear complexity with respect to the number of clients and parameters, i.e. $\mathcal{O}(MP)$ per communication round. We emphasize that the additional components introduced by the FCIL framework—such as shared or client-specific bias layers for incremental class correction—*do not affect the asymptotic complexity*. Their cost is negligible compared to the standard local training, since the number of parameters in these layers is small relative to the full model.

IV. Experimental Setup

In this section, we describe the datasets leveraged for the experiments (Sec. IV-A), along with the DL architecture employed (Sec. IV-B). Then, we introduce the FCIL scenarios evaluated in this work (Sec. IV-D) and the metrics to quantify NIDS capability to recognize and classify attacks (Sec. IV-E).

A. Network Traffic of Cyber-attacks

Datasets. For the experiments, we leverage 3 datasets including IoT datasets, collected from 2019 onward—i.e., `TON_IoT` [47], `IoT-NID` [48], and `Edge-IIoTset` [49].

They originally include *five categories* of attack [49]: (*i*) (Distributed) Denial of Service—(D)DoS—which includes attacks aiming at causing service disruptions; (*ii*) Info Gathering, which covers attacks focused on obtaining information from the victim; (*iii*) Man-in-the-Middle—MITM—addressing attacks involving the interception of communication; (*iv*) Injection, targeting the exploitation of vulnerabilities in applications or systems to introduce malicious data or code; (*v*) Malware, covering uncategorized attacks.

Within this category, various *sub-categories* can be identified based on the specific type of attack or the protocol involved [16]. For instance, in the DoS category, most datasets (except `TON_IoT`) include UDP floods, while HTTP and SYN floods are common across all. `TON_IoT` uniquely has a generic TCP flood; ACK and ICMP floods are specific to `IoT-NID` and `Edge-IIoTset`, respectively. Info gathering attacks mostly involve various scans: port (`Edge-IIoTset`, `IoT-NID`, `TON_IoT`), OS (`IoT-NID`, `Edge-IIoTset`), vulnerability (`Edge-IIoTset`), and host (`IoT-NID`). MITM attacks include ARP spoofing across datasets, with ICMP and DHCP spoofing in `TON_IoT` and DNS spoofing in `Edge-IIoTset`. Injection attacks cover SQL, XSS (`TON_IoT`, `Edge-IIoTset`), and HTTP injection (`Edge-IIoTset`). Lastly, malware includes brute-force attempts on HTTP (`TON_IoT`, `Edge-IIoTset`), telnet (`IoT-NID`), FTP (`TON_IoT`), plus backdoors and ransomware in `TON_IoT` and `Edge-IIoTset`. Concerning the physical testbed, `IoT-NID` leverages smart devices and general-purpose hardware, while `TON_IoT` and `Edge-IIoTset` use hybrid setups with both physical and emulated devices.

To ensure consistent labeling across all datasets, we apply a rule-based preprocessing approach that refines label granularity. The labeling scheme follows a standardized format of `category-type(-subtype)`, promoting interpretability and structure. Defined categories include `dos` (denial-of-service), `infog` (information gathering), `mitm` (man-in-the-middle), `inject` (injection), and `malwr` (malware) [16]. Furthermore, the majority classes of all three datasets are subjected to random sampling to achieve a biflow number ranging from 10k to 100k. Notably, when analyzing the most and least represented attack classes, the imbalance ratios—i.e., the ratio between the number of biflows of the most populous class over the least one—are 400, 700, and 340 for `TON_IoT`, `Edge-IIoTset`, and `IoT-NID`, respectively. This indicates a significant class imbalance—or skewness in the

biflow distribution—that persists even after undersampling, although it has been partially mitigated. The distributions of the biflows, resulting from the relabeling procedure, for the three considered datasets are shown in Fig. 2.[2]

DL Input. In this work, we consider 6 per-packet characteristics, i.e. Payload Length (PL), Inter Arrival Time (IAT), Packet Direction (DIR), TCP Window Size (WIN), TCP Flags (FLG), and Time To Live (TTL). To clarify, PL indicates the packet length, IAT refers to the time interval between successive packets, DIR denotes packet direction (0 for downstream, 1 for upstream), WIN represents the TCP window size (set to 0 for UDP packets), FLG stands for the TCP flags converted into a binary representation (ranging in $[0, 255]$), and TTL means how many routers (hops) a packet can pass through before being discarded. Fields such as IP addresses and ports are deliberately excluded from the model's training to prevent unwarranted bias [50]. In the following experiments, we use different feature sets: *base*—composed of PL, IAT, DIR, and WIN—and *extended*—adding also FLG and TTL. The latter features—i.e., TTL and FLG—have been shown to carry valuable information for distinguishing between benign and malicious traffic [51], and we aim to evaluate their utility for intrusion detection in a FCIL setting when focusing on the IoT ecosystem.

Last but not least, to support *early detection* of malicious traffic, we extract features from the first N packets of each biflow. Prior work [50, 52] has demonstrated that analyzing per-packet features from just the initial 10 packets yields strong classification results, with only marginal improvements when incorporating the entire biflow [53]. Therefore, time series data comprising the selected features from the first 10 packets of each biflow are extracted. For shorter biflows, zero-padding is applied to keep the sequence length consistent.

B. DL model architecture

We use a 2D-CNN architecture—originally introduced in [54] and widely adopted for both network Traffic Classification (TC) [50] and intrusion detection [15]—with modifications to support incremental learning. Importantly, our approach is model-agnostic and can be integrated with other DL architectures without requiring structural changes.

The `2D-CNN` is composed of two convolutional layers (`Conv2D`) followed by pooling and normalization operations, with a final `Dense` layer providing the classifier output (i.e., a confidence vector). The architecture has 540k trainable parameters and is fed with a (matrix-arranged) input $\boldsymbol{x}$ of *packets* $\times$ *fields* for each biflow.

At each incremental step, the backbone of the network remains structurally unchanged—i.e., no architectural modifications are introduced—while the head is expanded with neurons for new attacks introduced that episode by federated clients. Incremental training begins with the weights obtained from the previous incremental episode—i.e., the base model weights ($\boldsymbol{\theta}^0_{avg}$) for the initial one. At the end of each episode, all clients synchronize to a common model distributed by the federated server.

Additionally, to assess the effectiveness of our solution across a distinct DL architecture, we evaluate a hybrid model combining 2D-CNN and LSTM layers—denoted `HYBRID` in the following sections [54]. The network operates on the same input representation used by the 2D-CNN.

[2]We leverage bidirectional flows (viz., biflows) as the traffic object. It encompasses all packets that share the same quintuple (`src_ip`, `dst_ip`, `src_port`, `dst_port`, and `L4_proto`), with interchangeable `src` and `dst`.

Table II: FCIL scenarios for the three different datasets, detailed in the number of $\mathcal{K}^{old}$, $\mathcal{K}^{new}$, and clients.

Scenario	Dataset	$\lvert\mathcal{K}^{old}\rvert$	$\lvert\mathcal{K}^{new}\rvert$	N
S_1	TON_IoT	B + 9A	2A	2
	IoT-NID	B + 7A	2A	2
	Edge-IIoTset	B + 11A	2A	2
S_2	TON_IoT	B + 6A	5A	5
	IoT-NID	B + 4A	5A	5
	Edge-IIoTset	B + 8A	5A	5

A: attack class; B: benign class.

C. Upper Bound Approaches

To recap, FCIL introduces the incremental learning paradigm into a federated setup. Therefore, to measure separately the performance loss introduced by each challenge—i.e., decentralized data and need to update NIDS to cope with evolving threats—, we consider *two different upper bounds*: (*i*) `Scratch`, meaning that the model is trained *from-scratch* with all the data in a *centralized fashion* and *no incremental learning*, and (*ii*) *centralized CIL*, which involves an incremental training performed in a centralized setup—i.e., the new classes of the considered scenario are added on a single client.

While `Scratch` uses all the data at a single point to perform *training from scratch*, centralized CIL assumes that each client has access to $\mathcal{D}^{mem}$ for old classes along with the traffic data for the new classes it intends to add.

D. FCIL scenarios

For the experiments, we define different federated incremental scenarios. In detail, a scenario is compactly referred to as $\langle|\mathcal{K}^{old}|+|\mathcal{K}^{new}|, \mathcal{N}\rangle$ where $|\mathcal{K}^{old}|$ and $|\mathcal{K}^{new}|$ are the number of old and new classes, respectively, and $\mathcal{N}$ the number of clients in the federated network. Furthermore, in both scenarios S_1/S_2 we assume that each federated client observes a (different) new attack at a given time and cooperates with other clients to incorporate all the attacks seen by the group (i.e. in such a case it holds $|\mathcal{K}^{new}|= \mathcal{N}$). Additionally, all clients in the federated network adhere to a common agreement on the number of communication rounds ($\mathcal{R}$) to be performed. In each communication round, clients transmit their locally trained models to the server, which aggregates them into a global model and redistributes it to all clients.

The FCIL scenarios used for the experiments are reported in Tab. II, detailing for each dataset the number of old and new classes and the number of clients in the federated network. We consider federations of 2 and 5 clients, modeling realistic cross-silo settings in which groups of IoT devices (e.g., organizations or edge clusters) collaboratively train without sharing

raw data, rather than large-scale cross-device FL. Across all scenarios, $\mathcal{K}^{old}$ always includes the benign class, whereas $\mathcal{K}^{new}$ contains only attack classes, reflecting the decentralized emergence of new attacks.

E. Evaluation Metrics

In this work, we evaluate federated and incremental NIDS from two complementary perspectives: (i) their ability to classify network traffic by identifying both benign device activity and specific attack types, and (ii) their effectiveness in distinguishing legitimate traffic from malicious traffic.

To assess attack detection and recognition capabilities, we leverage *F1 Score* (briefly, F1) as performance metric [34]. It is computed as the harmonic mean of the per-class precision and recall and then we obtain its *macro version* by averaging the per-class metrics over all the classes, namely

$$\text{F1 Score}(\boldsymbol{\theta}, \mathcal{K}) = \frac{1}{|\mathcal{K}|} \sum_{i \in \mathcal{K}} \frac{2 \cdot Precision_i \cdot Recall_i}{Precision_i + Recall_i}, \quad (7)$$

where $\mathcal{K}$ denotes the generic class set and $\boldsymbol{\theta}$ a generic model (learned from-scratch or in federated incremental fashion).

To focus on NIDS detection capabilities, we use the partial area under the curve (pAUC), which emphasizes performance at low False Positive Rate (FPR). The pAUC is defined as:

$$\text{pAUC} = \int_{p_\ell}^{p_u} \text{TPR}(\text{fpr})\, d\text{fpr} \quad (8)$$

where $p_\ell = 0$ and $p_u = 0.01$, i.e., we compute the area under the ROC—defined by True Positive Rate (TPR)—up to 1% FPR, ensuring relevance for real-world NIDS conditions.

To obtain insights into both *catastrophic forgetting* and *intransigence* phenomena (due to the incremental learning process), we break down both the considered metrics by the subset of classes $\mathcal{K}^{old}$, $\mathcal{K}^{new}$, and $\mathcal{K}^{all}$, obtaining for a generic metric X the following sub-metrics: X^{old}, X^{new}, and X^{all}.

To quantify a model's gap from upper bound approaches, we define *drop from scratch* (`Drop-Scratch`) and *drop from CIL* (`Drop-CIL`) metrics measuring the difference in F1 and PAUC between FCIL NIDS and the upper bounds.

F. Implementation Details

In this section, we provide the implementation details for the FCIL framework used in this work. Specifically, we extend a Python-based framework—i.e., `FACIL`[3]—to handle federated learning training. In detail, we devise a federated server responsible for aggregating the models and coordinating and communicating with the clients during the local training procedure CIL. Additionally, we modify this framework to train multiple clients simultaneously, with each adding a new attack. The hyperparameters used for the training phase are the following: 200 epochs with patience of 20, an initial learning rate of 0.1, a learning rate decay factor of 3, and a minimum learning rate of 10^{-4}. For memory-based CIL approaches, 25 biflows per each $\mathcal{K}^{old}$ are selected [38]. In our experiments, we explore various synchronization frequencies by setting

[3]https://github.com/mmasana/FACIL

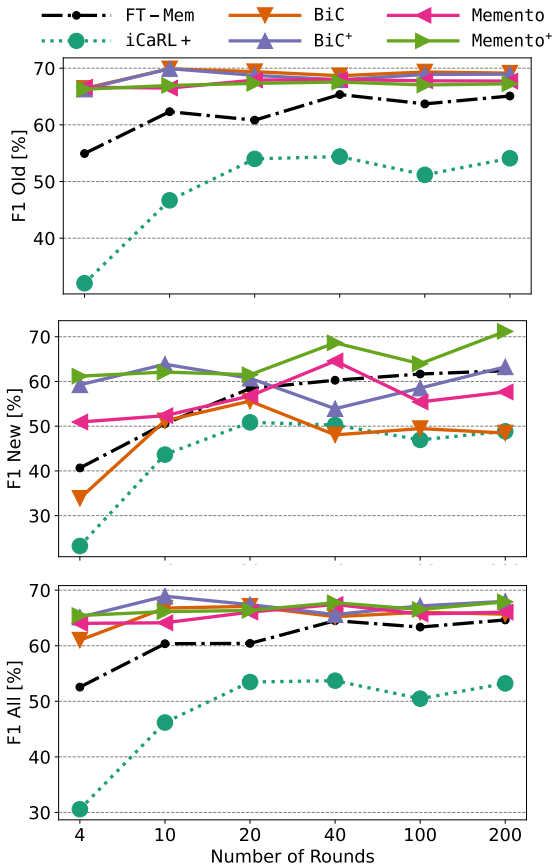


Figure 3: FCIL performance on `TON_IoT` in the $\mathcal{S}_1$ scenario with the base feature set. Different line styles distinguish approaches according to the mitigation strategies employed.

$\mathcal{R} = \{4, 10, 20, 40, 100, 200\}$. Each client conducts a total of 200 local training epochs, sharing its model with the server every $\mathcal{E} = 200/\mathcal{R}$ epochs. For each configuration, we test 10 seeds, corresponding to 10 distinct $(7 \div 3)$ train-test splits. Results are shown as the average across the incremental runs using 10 different seeds, each adding different new classes.

The rest framework leverages NumPy[4] and Pandas[5] for data management and pre-processing, PyTorch[6] for training and evaluating NIDS, and Matplotlib[7] and Seaborn[8] to plot results.

V. Experimental Results

We begin by analyzing performance in a few-client setting, optimizing FCIL along two axes: feature set and FL algorithm (Sec. V-A). Then, we scale to a larger number of clients (Sec. V-B), compare FCIL with *training from scratch* and centralized CIL (Sec. V-C). Furthermore, we evaluate the generalizability of our FCIL solution across different DL architectures (Sec. V-D) and examine the impact of IID attack data among clients (Sec. V-E). Lastly, we assess real-world

[4]https://numpy.org/
[5]https://pandas.pydata.org/
[6]https://pytorch.org/
[7]https://matplotlib.org/
[8]https://seaborn.pydata.org/

Table III: FCIL performance on TON_IoT in the $\mathcal{S}_1$ scenario with *expanded feature set*. Results are shown as the difference of F1 w.r.t. the *base feature set*. Furthermore, they are broken down by the number of synchronization rounds performed and the set of classes. **Green boldface text** indicates improvement over the base set of features, while **red one** refers to decay in performance. *Gray background* highlights for each approach the $\mathcal{R}$ at which the best improvement over the base feature set is obtained, while the *green ones* highlight the best approach and the best $\mathcal{R}$ for the considered setup.

CIL Appr.	$\mathcal{R}$=4			$\mathcal{R}$=10			$\mathcal{R}$=20			$\mathcal{R}$=40			$\mathcal{R}$=100			$\mathcal{R}$=200		
	$F1^{old}$	$F1^{new}$	$F1^{all}$	$F1^{old}$	$F1^{new}$	$F1^{all}$	$F1^{old}$	$F1^{new}$	$F1^{all}$	$F1^{old}$	$F1^{new}$	$F1^{all}$	$F1^{old}$	$F1^{new}$	$F1^{all}$	$F1^{old}$	$F1^{new}$	$F1^{all}$
FT-Mem	+8.78	+10.85	+9.13	+4.92	+2.71	+4.55	+13.80	+15.31	+14.05	+10.38	+8.53	+10.07	+11.81	+9.07	+11.35	+10.97	+3.47	+9.72
iCaRL+	+2.91	+2.92	+2.91	+7.17	+5.18	+6.84	+8.21	+4.31	+7.56	-17.52	-13.09	-16.78	-6.71	-5.74	-6.55	-17.55	-13.20	-16.82
BiC	+3.40	+4.82	+3.64	+6.70	-2.12	+5.23	+8.66	+1.97	+7.55	+8.90	+8.07	+8.76	+8.51	+4.43	+7.83	+8.13	+3.24	+7.32
MEMENTO	+9.86	-5.98	+7.22	+10.52	+9.03	+10.27	+9.89	-1.63	+7.97	+8.84	-16.33	+4.64	+10.12	-2.39	+8.03	+8.98	+1.35	+7.71
BiC$^+$	+3.15	+0.32	+2.68	+6.57	+1.42	+5.71	+8.40	+2.49	+7.41	+7.42	+7.48	+7.43	+7.76	+5.88	+7.45	+8.13	+1.43	+7.01
MEMENTO$^+$	+9.53	+4.68	+8.72	+9.55	+5.52	+8.88	+9.98	+4.65	+9.09	+8.45	-8.17	+5.68	+10.46	+2.83	+9.19	+8.92	-3.82	+6.79

applicability, focusing on cross-network generalization and few-sample learning (Sec. V-F).

A. Few-client scenario

Hereinafter, we evaluate FCIL performance in the $\mathcal{S}_1$ scenario using TON_IoT. Each paragraph answers a key research question, stated as its title.

How effective is FedAvg with a base set of features? In response, we evaluate all CIL methods combined with FedAvg—using *base feature set* as input (viz., PL, IAT, DIR, and WIN). Results in Fig. 3 show that performance on $\mathcal{K}^{all}$ closely mirrors that of $\mathcal{K}^{old}$, largely due to the imbalanced scenario with 10 base classes and only 2 new ones (attacks).

Performance trends vary across methods, depending on their CIL-focused mitigation strategies. Using only rehearsal (i.e., FT-Mem), F1 generally improves with more rounds, peaking at $\mathcal{R}=200$ (i.e., 65% $F1^{all}$). In contrast, iCaRL+ (combining rehearsal and distillation) reaches its best performance earlier, at $\mathcal{R}=40$ (≈54% $F1^{all}$/$F1^{old}$ and ≈50% $F1^{new}$). As FT-Mem, iCaRL+ struggles at low $\mathcal{R}$ but benefits from higher rounds.

CIL methods using all three mitigation strategies *behave differently*. Their performance on $\mathcal{K}^{old}$ drops as $\mathcal{R}$ increases. For $\mathcal{K}^{new}$, outcomes depend on the bias correction strategy: (*i*) BiC—i.e., with a single shared correction layer for all new classes—achieve peak $F1^{new}$ scores at $\mathcal{R}=20$ (≈56%) and (*ii*) BiC$^+$ reaches its highest $F1^{new}$ (≈64%) at $\mathcal{R}$=10. MEMENTO behaves like BiC, peaking at $\mathcal{R}=40$. But MEMENTO$^+$ follows a different pattern: performance steadily rises with more rounds, reaching its best $F1^{new}$ (≈71%) at $\mathcal{R}$=200.

> **Takeaways.** Old-class performance remains stable even with limited synchronization, whereas new classes benefit from additional rounds to improve understanding. In addition, the proposed client-specific bias correction strategy boosts BiC$^+$/MEMENTO$^+$ effectiveness. Notably, the best trade-off is obtained by BiC$^+$ at $\mathcal{R}$=10 on $\mathcal{K}^{all}$ and $\mathcal{K}^{old}$ (≈69% $F1^{all}$, ≈70% $F1^{old}$), while MEMENTO$^+$ (second-best) reaches highest $F1^{new}$ (≈71%) at $\mathcal{R}$=200).

Does expanded feature set boost performance? Then, we evaluate the impact of using an *expanded* feature set. The results are reported in Tab. III, highlighting the performance improvement in **green** (resp. decay in **red**) obtained using the expanded set of features over the base one.

Generally speaking, performance on $\mathcal{K}^{old}$ highly benefits from the use of the expanded set of features, with FT-Mem obtaining the highest gain w.r.t. all the other approaches. Notably, it improves at most by ≈14% at $\mathcal{R}$=20. Performance trends on $\mathcal{K}^{all}$ closely reflect those observed for old classes.

On the other hand, $\mathcal{K}^{new}$ presents more unstable trends. Specifically, FT-Mem, BiC, and BiC$^+$ demonstrate a generally positive trend across almost all synchronization values, with improvements of up to 15%. In contrast, MEMENTO and MEMENTO$^+$ experience a critical drop at $\mathcal{R}$=40, losing 16% and 8%, respectively, compared to the base feature set.

It is worth noting that iCaRL+ exhibits a distinctive trend. Its performance improves steadily by up to ≈8% as the synchronization value increases to $\mathcal{R}$=20, but beyond this point, it experiences a sharp decline, dropping to around −18%. This behavior may stem from the iCaRL+ loss becoming less effective at higher synchronization intervals, potentially causing forgetting or training instability.

> **Takeaways.** Expanded features lead to a performance improvement across all the synchronization round values (except for iCaRL+), by stabilizing old classes performance, but they can destabilize new-class learning in MEMENTO/MEMENTO$^+$ at mid synchronizations. In summary, MEMENTO$^+$ obtains the best performance at $\mathcal{R}=10$ (≈ 75% $F1^{all}$, ≈ 76% $F1^{old}$, and ≈ 70% $F1^{new}$).

What is the impact of different FL algorithms on performance? Lastly, we evaluate the performance of different FL algorithms, focusing on the top-2 CIL approaches—i.e., BiC$^+$ and MEMENTO$^+$—and using the expanded feature set as input. Results of this analysis are shown in Fig. 4.

Notably, FedProx shows significant improvements over FedAvg for both BiC$^+$ and MEMENTO$^+$, especially with a low number of synchronizations. As $\mathcal{R}$ increases, performance of FedAvg and FedProx converges with both scoring between 74% and 76% $F1^{all}$ for BiC$^+$ and MEMENTO$^+$, respectively. Performance convergence is due to the improvement of FedAvg over FedProx on $\mathcal{K}^{old}$ with higher $\mathcal{R}$, while FedProx delivers better performance on $\mathcal{K}^{new}$, especially with MEMENTO$^+$. However, both approaches exhibit a declining trend on $\mathcal{K}^{old}$ and $\mathcal{K}^{new}$ performance as $\mathcal{R}$ increases.

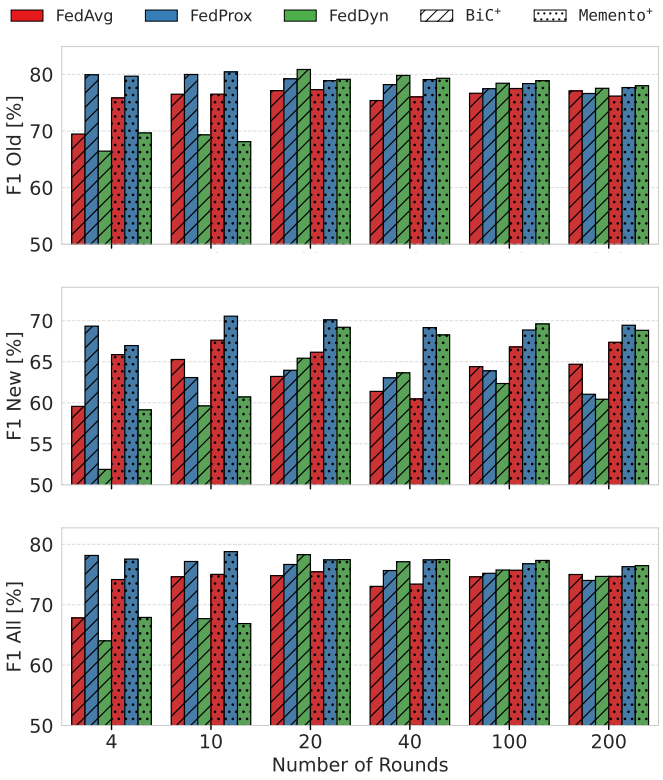


Figure 4: Comparison of different FL approaches for TON_IoT with expanded feature set in the $\mathcal{S}_1$ scenario.

On the other hand, FedDyn shows lower performance for low $\mathcal{R}$, while obtaining the highest F1 for $\mathcal{R} \geq 20$. This behavior can be attributed to the dynamic regularization in FedDyn, which corrects for client drift by incorporating a linear term related to the global model. Similar to FedAvg, as $\mathcal{R}$ increases, FedDyn shows improved F1^{old}, peaking at $\mathcal{R} = 20$, and F1^{new}, with best value occurring at $\mathcal{R} = 100$ (resp. $\mathcal{R} = 20$) for MEMENTO$^+$ (resp. BiC$^+$).

Takeaways. FCIL stability–plasticity is strongly influenced by the choice of FL algorithm: FedProx supports rapid early integration, FedAvg trails behind across most settings, and FedDyn overtakes the others only with many synchronization rounds. The best trade-off of performance is reached by FedProx using MEMENTO$^+$ at $\mathcal{R}$=10, achieving ≈79% F1^{all}, ≈80% F1^{old}, and ≈71% F1^{new}.

B. Many-client scenario

In this section, we evaluate FCIL approaches in the $\mathcal{S}_2$ scenario for the TON_IoT dataset with the *expanded feature set*. Table IV reports the best trade-off achieved for each combination of CIL and FL at specific values of $\mathcal{R}$. It is worth noting that having more clients with new classes makes this scenario more challenging, as more knowledge needs to be integrated into the model at each synchronization. Overall, a few synchronization rounds significantly hinder new knowledge acquisition, highlighting the need for frequent updates as client number and new classes grow.

Table IV: FCIL performance on TON_IoT in the $\mathcal{S}_2$ scenario using the *expanded feature set*. For each combination, the results highlight the best synchronization value, corresponding to the highest achieved F1^{all}.

FL Appr.	CIL Appr.	optimal $\mathcal{R}$	F1 Score [%]		
			Old	New	All
FedAvg	BiC$^+$	100	76.38	66.48	72.25
	MEMENTO$^+$	20	79.64	64.32	73.25
FedProx	BiC$^+$	40	78.97	67.47	74.18
	MEMENTO$^+$	100	79.05	68.10	74.49
FedDyn	BiC$^+$	200	79.28	65.13	73.38
	MEMENTO$^+$	100	78.68	66.83	73.74

While no strict trends emerge, F1^{new} generally increases and F1^{old} decreases as $\mathcal{R}$ grows. Similar to $\mathcal{S}_1$, FedDyn performs notably worse at $\mathcal{R} = \{4, 10\}$, whereas FedProx consistently leads at higher $\mathcal{R}$, followed by FedDyn. Notably, FedProx preserves old knowledge comparably to FedAvg and FedDyn, while achieving better F1^{new}, making it the best trade-off among the three.

Takeaways. Many clients make FCIL's process more challenging, requiring more synchronizations to include new knowledge. With a larger number of clients (i.e., $\mathcal{S}_2$), MEMENTO$^+$ in conjunction with FedProx results in the top-performing FCIL approch, achieving the highest F1^{all} at $\mathcal{R} = 100$ (≈75%), along with ≈80% F1^{old} and ≈68% F1^{new}.

C. Wrap-up of results

In this section, we compare the performance of the top-performer FCIL approach against the upper bound Scratch baseline and centralized CIL training. Specifically, we evaluate performance in solving two relevant NIDS tasks: (i) bMD—i.e., distinguishing among legitimate and malicious traffic (all attacks as one class); (ii) AC—i.e., fine-grained classification of malicious traffic into different attack types. Table V summarizes the results for the two scenarios, across all three evaluated datasets and the corresponding tasks discussed above.

Considering scenario $\mathcal{S}_1$, the best FCIL approach (i.e., MEMENTO$^+$ with FedProx at $\mathcal{R} = 10$) *consistently outperforms* the best centralized CIL method (i.e., MEMENTO) across bMD/AC tasks for both TON_IoT and Edge-IIoTset datasets. However, despite these improvements, our solution still lags behind the Scratch baseline, with performance gaps reaching up to 3% in bMD and 8% in AC. In contrast, for the IoT-NID dataset, the CIL method slightly outperforms the FCIL one, showing improvements of +1% in PAUC^{All} and +2% in F1^{all} for AC. Moreover, the gap between our method and Scratch is even more pronounced for IoT-NID, with differences of up to 15% in AC. The ROC curves in Fig. 5 reveal a notable decline in TPR for the bMD task when compared to the Scratch baseline—particularly at low false positive rates (FPR≤1%). This drop affects both CIL and FCIL approaches, which

Table V: Summary of performance for the best FCIL approach—in centralized and distributed fashion—and Scratch, across scenarios (i.e., S_1 and S_2) and datasets (i.e., TON_IoT, IoT-NID, and Edge-IIoTset). Performance is reported as absolute value and drop w.r.t. Scratch and CIL approaches. **Red** (resp. **green**) text highlights the gap (resp. improvement) compared to upper bounds (i.e., Scratch and CIL).

Scenario	Dataset	Approach	bMD			AC		
			pAUC^{Old}	pAUC^{New}	pAUC^{All}	F1^{Old}	F1^{New}	F1^{All}
$\mathcal{S}_1$	TON_IoT	Scratch	99.05	99.48	99.13	83.68	92.61	85.30
		Best CIL	92.67	90.53	92.28	76.80	70.85	75.72
		Best FCIL*	96.77	91.57	95.82	81.20	75.11	80.09
		Drop-Scratch/ CIL	**2.28 / -4.10**	**7.91 / -1.04**	**3.31 / -3.54**	**2.48 / -4.40**	**17.50 / -4.26**	**5.21 / -4.37**
	Edge-IIoTset	Scratch	99.96	99.96	99.96	76.19	87.12	77.87
		Best CIL	98.16	99.80	98.41	62.84	53.35	61.38
		Best FCIL*	98.70	98.65	98.69	68.49	74.78	69.46
		Drop-Scratch/ CIL	**1.26 / -0.54**	**1.31 / 1.15**	**1.27 / -0.28**	**7.70 / -5.65**	**12.34 / -21.43**	**8.41 / -8.08**
	IoT-NID	Scratch	96.27	96.68	96.36	98.48	98.51	98.48
		Best CIL	93.45	92.12	93.15	89.54	71.39	85.51
		Best FCIL*	93.21	86.58	91.74	86.71	70.98	83.21
		Drop-Scratch/ CIL	**3.06 / 0.24**	**10.10 / 5.54**	**4.62 / 1.41**	**11.77 / 2.83**	**27.53 / 0.41**	**15.27 / 2.30**
$\mathcal{S}_2$	TON_IoT	Scratch	99.08	99.19	99.13	83.32	87.69	85.30
		Best CIL	92.59	91.64	92.16	68.24	77.69	72.57
		Best FCIL†	96.86	86.44	92.12	79.13	75.99	77.70
		Drop-Scratch/ CIL	**2.22 / -4.27**	**12.75 / 5.20**	**7.01 / 0.04**	**4.19 / -10.89**	**11.70 / 1.70**	**7.60 / -5.13**
	Edge-IIoTset	Scratch	99.96	99.98	99.96	71.17	88.58	77.87
		Best CIL	94.65	90.64	93.11	53.43	70.35	59.94
		Best FCIL†	98.54	93.42	96.57	62.77	57.06	60.57
		Drop-Scratch/ CIL	**1.42 / -3.89**	**6.56 / -2.78**	**3.39 / -3.46**	**8.40 / -9.34**	**31.52 / 13.29**	**17.30 / -0.63**
	IoT-NID	Scratch	96.08	96.60	96.36	98.39	98.55	98.48
		Best CIL	89.78	88.85	89.26	90.82	88.02	89.27
		Best FCIL†	94.12	87.22	90.29	88.79	78.77	83.22
		Drop-Scratch/CIL	**1.96 / -4.34**	**9.38 / 1.63**	**6.07 / -1.03**	**9.60 / 2.03**	**19.78 / 9.25**	**15.26 / 6.05**

Best CIL: MEMENTO; **Best FCIL**: MEMENTO$^+$ + FedProx with $\mathcal{R}$=10 (*); MEMENTO$^+$ + FedProx with $\mathcal{R}$=100 (†).

exhibit comparable patterns. The degradation is especially severe for $\mathcal{K}^{new}$, as reflected in the steeper decline of the ROC curves at low FPR values. Notably, MEMENTO$^+$ consistently outperforms MEMENTO, achieving higher TPR across both $\mathcal{K}^{old}$ and $\mathcal{K}^{all}$ for equivalent FPR thresholds.

In scenario $\mathcal{S}_2$, we observe a greater ability of the FCIL approach to retain previously learned knowledge compared to the CIL counterpart, as reflected in consistently higher performance across all class-level metrics. However, FCIL shows notable difficulty in incorporating new attack classes, with significant performance drops relative to CIL—reaching up to 13% for Edge-IIoTset in the AC task. Increasing the number of clients further degrades performance for both FCIL and CIL, while also amplifying the gap to the Scratch baseline. For TON_IoT and Edge-IIoTset, the performance drops nearly double, peaking at 32% in DropF1^{New} and 17% in DropF1^{All}. In contrast, for IoT-NID, the degradation remains comparable to that observed in $\mathcal{S}_1$, with the DropF1^{New} in the AC task even slightly lower than in the previous scenario.

Takeaways. FCIL enables robust decentralized incremental learning but lags behind centralized CIL and Scratch. However, in both scenarios, FCIL outperforms CIL on TON_IoT and Edge-IIoTset. Conversely, on IoT-NID, FCIL struggles relative to CIL and Scratch, likely due to the presence of more difficult or rarer attacks as new classes.

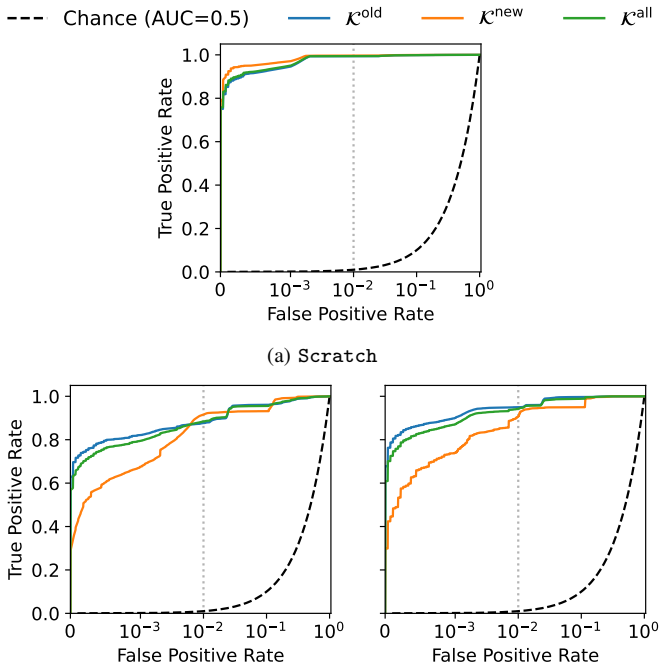
Table VI: Multiclass misuse detection performance for the best FCIL approach, obtained using the HYBRID architecture [54], compared by upper bounds—i.e., Scratch, and CIL.

Scenario	Approach	mMD		
		F1^{Old}	F1^{New}	F1^{All}
$\mathcal{S}_1$	Scratch	85%	88%	86%
	Best CIL	75	60	72
	FCIL	80	72	79%
	Drop-Scratch/ CIL	**5/-5**	**16/-12**	**7/-7**
$\mathcal{S}_2$	Scratch	85	88	86
	Best CIL	63	52	59
	FCIL	78	60	70
	Drop-Scratch/ CIL	**7/-12**	**28/-8**	**16/-11**

D. Generalization across DL architectures

To assess the architecture-independent nature of our solution, we evaluate the mMD performance for TON_IoT when using the HYBRID network. Table VI shows results both for few-client ($\mathcal{S}_1$) and many-client ($\mathcal{S}_2$) scenarios, additionally comparing FCIL with upperbound approaches—namely, CIL and Scratch.

Results are consistent with those reported in Tab. V for TON_IoT. Notably, the FCIL approach consistently outperforms centralized CIL when using the HYBRID architecture. In the $\mathcal{S}_1$ scenario, FCIL yields improvements of 5% F1 on previously seen classes and 12% F1 on new classes, for an overall gain of +7% in F1^{all}. Similarly, in $\mathcal{S}_2$, FCIL better retains prior knowledge (+12% F1) and incorporates new classes more effectively (+8% F1), resulting in an average



(a) `Scratch`

(b) `MEMENTO`—CIL fashion

(c) `MEMENTO`$^+$—FCIL fashion

Figure 5: ROC curves for (a) `Scratch`, (b) `MEMENTO`, and (c) `MEMENTO`$^+$ in the $\mathcal{S}_1$ scenario on the `TON_IoT` dataset. Curves are averaged across all the seeds. The x-axis is displayed *in a symlog scale* for improved visibility.

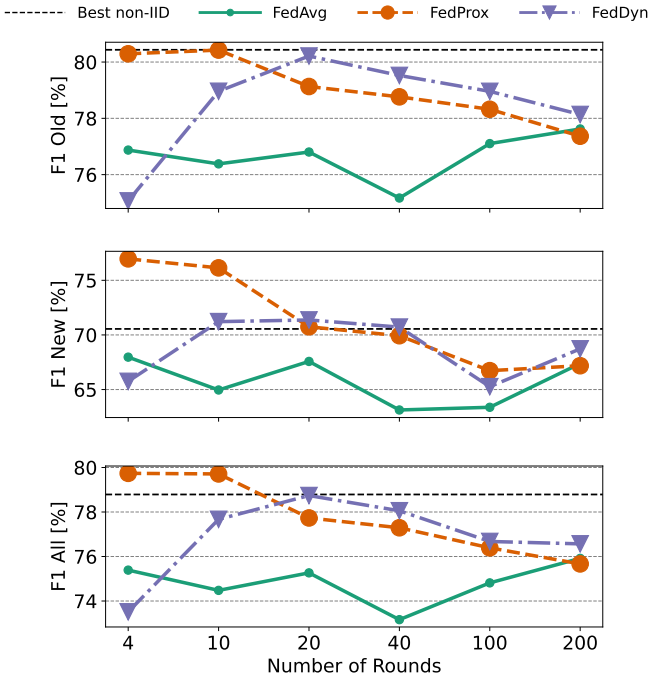


Figure 6: Comparison of different FL approaches for `TON_IoT` on new IID data, alongside the best non-IID configuration.

performance increase of 11% F1 over centralized CIL.

Despite these advantages, FCIL remains below the ideal `Scratch` reference, particularly as the number of clients grows. In $\mathcal{S}_1$, it incurs a DropF1^{New} of 16% and a DropF1^{Old} of 5%, whereas in $\mathcal{S}_2$ these losses rise to 12% and 28%, corresponding to F1^{all} reductions of 7% and 16%, respectively.

> **Takeaways.** FCIL delivers consistent gains over centralized CIL even when shifting to a different DL architecture, confirming that the benefits of our approach generalize beyond a single model design.

E. NIDS under new IID data

In this section, we evaluate the performance of FCIL NIDS under an IID data, focusing on a few-client scenario where both clients encounter the same two novel attacks, but with distinct biflows. Results of this analysis are reported in Fig. 6 for the different FL algorithms, along with the best FCIL configuration in the non-IID setup in which each client encounters a disjoint new attack, obtained in Sec. V-A.

Similarly to the non-IID configuration, best results are yielded also in the IID setup with low R, then performance decreases when the communication rate exceeds a given threshold. This behavior suggests that, under IID conditions, additional communication rounds provide diminishing returns once the models have sufficiently converged.

Comparing FL approaches, `FedProx` and `FedDyn` consistently outperform `FedAvg` across most settings. This indicates that regularization-based approaches remain beneficial even when client data are IID. Notably, `FedDyn` generally achieves the best or near-best performance for intermediate values of R (i.e., $R = 20$), highlighting its robustness in maintaining previously-learned knowledge, while `FedProx` turns out to be the best with low R—namely, $R=4$.

Across all methods, performance on the $\mathcal{K}^{new}$ split is systematically lower than on $\mathcal{K}^{old}$, regardless of R. This gap reflects the inherent difficulty of CIL (even in the absence of data heterogeneity) and suggests that catastrophic interference with newly-introduced attacks remains a central challenge. Nevertheless, moving from a non-IID to an IID setting yields a substantial benefit for learning new classes, with the best IID configuration—i.e., `FedProx` at $R=4$—improves performance on $\mathcal{K}^{new}$ by $\approx$10% F1 compared to the best non-IID result—i.e., `FedProx` at $R = 10$. This highlights the impact of data distribution on plasticity and confirms that data homogeneity significantly alleviates, though does not eliminate, the challenges associated with incremental class acquisition.

> **Takeaways.** In IID settings, FCIL performance peaks at low communication rounds, with diminishing returns as R increases; `FedProx` outperform other FL algorithms, and data homogeneity yields up to a 10% F1 gain on $\mathcal{K}^{new}$ compared to the best non-IID setup, despite persistent challenges in including new knowledge.

F. NIDS in the Wild: Practical Challenges

In this section, we analyze some practical challenges of NIDS when deployed in real-world scenarios: (*i*) capability

Table VII: Binary detection capabilities in $\mathcal{S}_1$ as PAUC^{All} for each $\langle source, target \rangle$ pair for the best FCIL solution and the best CIL one (in brackets). Green background highlights the pairs for which $source = target$.

Source Dataset	Target Dataset		
	TON_IoT	Edge-IIoTset	IoT-NID
TON_IoT	95.82 (92.28)	74.90 (69.42)	57.33 (52.58)
Edge-IIoTset	54.88 (52.79)	98.69 (98.41)	60.60 (60.92)
IoT-NID	51.97 (55.59)	64.51 (51.68)	91.74 (93.15)

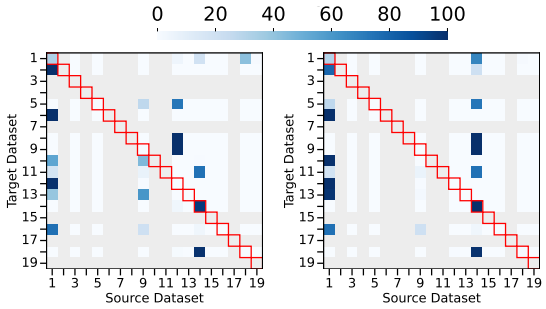
of generalizing across networks and (ii) ability to detect 0-day attacks with only a limited number of biflows available.

How well do FCIL NIDS generalize across diverse networks? In response, we separately evaluate the detection and classification performance of the best-performing FCIL model when deployed across different networks, and compare it with the best CIL model under the same conditions. Specifically, we report the results for the bMD task in Tab. VII, while Fig. 7 shows the AC performance when FCIL NIDS is trained on TON_IoT and tested with network traffic from Edge-IIoTset and IoT-NID. Other combinations are not shown for brevity.

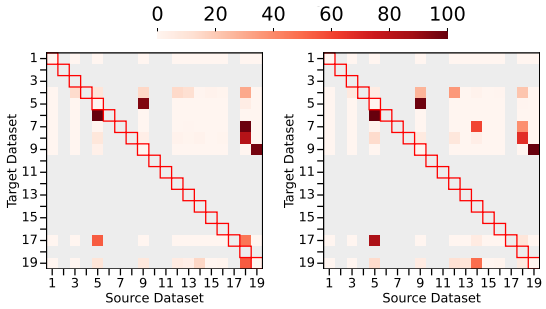
Specifically, Tab. VII clearly shows a decline in detection performance when the NIDS is evaluated on a different network from the one it was trained on. While *intra-dataset* models achieve $\geq 92\%$ PAUC^{All}, performance drops significantly in *cross-dataset* setup—reaching at most 75% PAUC^{All} (for $\langle$TON_IoT,Edge-IIoTset$\rangle$), with other cases falling below 60% PAUC^{All}. In most of the combinations, the FCIL approach outperforms the CIL baseline in both intra-dataset and cross-dataset scenarios, likely due to the decentralized training and different client-specific data distributions.

Similarly, the overall performance on the AC task remains unsatisfactory. We observe *distinct behavioral patterns* between the FCIL and CIL NIDS (see Fig. 7). Specifically, when using TON_IoT as the source dataset and Edge-IIoTset as the target, the FCIL setup shows a clear bias toward certain attack classes—such as dos-http, inject-sql, and malwr-backd-tcp—which are also the most frequent in the source dataset (Fig. 7a). In contrast, the CIL setup exhibits even more pronounced bias, particularly toward the dos-tcp and malwr-brutef-http classes (Fig. 7b). When IoT-NID is used as the target dataset, the trend reverses. Both approaches show a clear prediction bias toward specific attack classes—such as dos-tcp-syn, infog-scan-port, malwr-ransomware, and mitm-arpspoof (Fig. 7c). The CIL setup displays a similar bias, though it is less pronounced compared to the FCIL counterpart (Fig. 7d). Similar results are observed across the other datasets and scenarios; their confusion matrices are omitted for brevity.

Takeaways. FCIL NIDS generalize poorly across networks. Notably, cross-dataset performance are biased toward majority attacks within the training set. Despite decentralized training, detection performance remains below 65% PAUC.



(a) FCIL setup (b) CIL setup



(c) FCIL setup (d) CIL setup

N^o	Classes	N^o	Classes
		10	infog-scan-vuln
1	dos-http	11	inject-http
2	dos-icmp	12	inject-sql
3	dos-tcp	13	inject-xss
4	dos-tcp-ack	14	malwr-backd-tcp
5	dos-tcp-syn	15	malwr-brutef-ftp
6	dos-udp	16	malwr-brutef-http
7	infog-scan-host	17	malwr-brutef-telnet
8	infog-scan-os	18	malwr-ransomware
9	infog-scan-port	19	mitm-arpspoof

Figure 7: Confusion matrices for NIDS trained on TON_IoT (source dataset) and evaluated on Edge-IIoTset (top row) and IoT-NID (bottom row) in $\mathcal{S}_1$. Gray-shaded rows and columns indicate classes that are absent in the target or source dataset.

Can FCIL NIDS detect new 0-day attacks with just a few samples? In this analysis, we assess the ability of a FCIL NIDS to incorporate new attacks while retaining knowledge of previously learned ones, under a scenario characterized by a *scarcity of new attack samples*. Specifically, we constrain the number of biflows available for each new attack class to fixed values: $15, 25, 50, 100, 200$. For this analysis, we select for the few-shot training the seed that yields the best FCIL performance in the full-data setup ($+\inf$). For each setting, samples are randomly drawn from the complete dataset corresponding to that class. To ensure robustness and variability in sample selection, each experiment is repeated 10 times, with results expressed as the average across the repetitions.

Figure 8 compares the performance obtained by the best FCIL solution—under few-shot conditions (15 to 200 samples) and full data ($+\inf$)—and Scratch for the three considered datasets in the few- and many-client scenarios. In $\mathcal{S}_1$, performance on $\mathcal{K}^{old}$ is only slightly impacted by limited sample availability. Interestingly, performance on $\mathcal{K}^{new}$ is often comparable to—or even better than—the full-sample setting, except for Edge-IIoTset, where a notable drop is

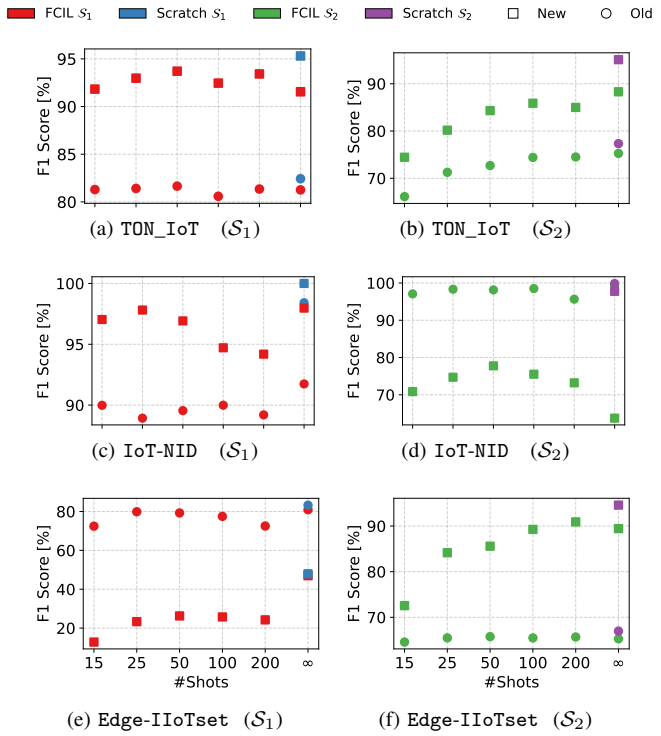


Figure 8: Performance under *few-shot conditions* (ranging from 15 to 200 samples) and *with all samples* ($+\infty$) for the best FCIL method and `Scratch`, in both few-client and many-client scenarios using the best seed.

observed as samples decrease. This may be due to higher sample variability or greater overlap in new attack fingerprints for the selected seed in `Edge-IIoTset`.

Similarly, in $\mathcal{S}_2$, $\mathcal{K}^{old}$ performance remains consistent between the two different settings. However, classification on $\mathcal{K}^{new}$ improves for `IoT-NID` and `Edge-IIoTset`, while `TON_IoT` scores similarly to the full-sample setting.

Takeaways. FCIL NIDS can learn new 0-day attacks from very few samples while preserving prior knowledge, often improving new-class recognition across client scenarios—e.g., `TON_IoT` and `IoT-NID` in $\mathcal{S}_1$ or `IoT-NID` and `Edge-IIoTset` in $\mathcal{S}_2$.

VI. Conclusion

Guaranteeing security is particularly challenging in IoT environments, where diverse devices continuously collect and exchange data. In this domain, ML- and DL-based NIDS are crucial for defending against cyberattacks. However, the dynamic nature of IoT networks—characterized by frequent device churn and heterogeneity—leads to a broader and more exposed attack surface. New attacks may emerge frequently, making NIDS models quickly outdated and requiring the ability to learn new threats incrementally. Furthermore, attack traffic can vary significantly across networks. While sharing data across deployments could improve detection coverage, privacy concerns make this approach impractical.

In this work, we bridge the gap between CIL—enabling continuous learning—and FL—leveraging distributed data for training—by proposing FCIL, combining both approaches to design an adaptable, collaboratively trained NIDS. Our results show that FCIL methods effectively retain prior knowledge, especially leveraging TCP flags and time-to-live as input, but face challenges integrating new attacks.

Overall, `MEMENTO`$^+$ combined with FedProx achieves the best results in both few- and many-client settings. In the former, fewer synchronization rounds ($\mathcal{R} = 10$) suffice for integrating new classes, while larger networks require more rounds ($\mathcal{R}$=100) to accommodate new knowledge. Under these conditions, `MEMENTO`$^+$ attains 79% and 75% F1^{all} in the few- and many-client scenarios on `TON_IoT`, respectively. Generally speaking, FCIL shows advantages over CIL in smaller networks, though it does not reach the performance of *training-from-scratch*. As the number of clients grows, FCIL better preserves existing knowledge but faces increased difficulty in learning new attacks, leading to larger performance drops.

In addition, we evaluate the NIDS in a *real-world deployment scenario*, assessing both their (i) generalization capabilities and (ii) their ability to learn from limited samples. Notably, deploying a NIDS in a different network environment significantly reduces its detection and classification performance. However, the FCIL NIDS consistently outperforms its CIL counterpart, demonstrating greater resilience to network shifts. On the other hand, in few-shot settings, FCIL NIDS achieves performance comparable to the full-data scenario—obtaining similar performance on old classes and sometimes even outperforming it on new ones.

Our *future directions* include (a) enhancing CIL approaches for better integration of new attacks in federated settings, (b) advancing personalization to balance local adaptation with global knowledge, (c) strengthening privacy protections, (d) reducing communication overhead, (e) improving robustness against adversarial threats, and (f) including federated domain-incremental learning.

Acknowledgment

This work is partially supported by the "xInternet" Project––funded by MUR—within the PRIN 2022 program (D.D.104-02/02/2022) and in part by the European Union under the Italian National Recovery and Resilience Plan (NRRP) of NextGenerationEU, partnership on "Telecommunications of the Future" (PE00000001 - program "RESTART"). This manuscript reflects only the authors' views and opinions and the Ministry cannot be considered responsible for them.

References

[1] "Microsoft Digital Defense Report: 600 million cyberattacks per day around the globe," https://news.microsoft.com/en-cee/2024/11/29/microsoft-digital-defense-report-600-million-cyberattacks-per-day-around-the-globe/, 2024, accessed: 2025-06-12.

[2] Cisco Systems, "Detecting zero-days with SnortML," https://www.cisco.com/c/en/us/products/collateral/security/firewalls/detecting-zero-days-with-snortml-wp.html, 2024, accessed: 2025-05-22.

[3] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis

et al., "Understanding the Mirai botnet," in *26th USENIX security symposium (USENIX Security 17)*, 2017, pp. 1093–1110.

[4] B. Zhao, S. Ji, J. Xu, Y. Tian, Q. Wei, Q. Wang, C. Lyu, X. Zhang, C. Lin, J. Wu *et al.*, "A large-scale empirical analysis of the vulnerabilities introduced by third-party components in IoT firmware," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 442–454.

[5] G. Apruzzese, L. Pajola, and M. Conti, "The cross-evaluation of machine learning-based network intrusion detection systems," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 5152–5169, 2022.

[6] S. Agrawal, S. Sarkar, O. Aouedi, G. Yenduri, K. Piamrat, M. Alazab, S. Bhattacharya, P. K. R. Maddikunta, and T. R. Gadekallu, "Federated learning for intrusion detection system: Concepts, challenges and future directions," *Computer Communications*, vol. 195, pp. 346–361, 2022.

[7] E. M. Campos, P. F. Saura, A. González-Vidal, J. L. Hernández-Ramos, J. B. Bernabé, G. Baldini, and A. Skarmeta, "Evaluating Federated Learning for intrusion detection in Internet of Things: Review and challenges," *Computer Networks*, vol. 203, p. 108661, 2022.

[8] H. Zhang, J. Ye, W. Huang, X. Liu, and J. Gu, "Survey of federated learning in intrusion detection," *Journal of Parallel and Distributed Computing*, p. 104976, 2024.

[9] M. Data and M. Aritsugi, "An incremental learning algorithm on imbalanced data for network intrusion detection systems," in *10th ACM International Conference on Computer and Communications Management (ICCCM)*, 2022, pp. 191–199.

[10] C. Oikonomou, I. Iliopoulos, D. Ioannidis, and D. Tzovaras, "A multi-class intrusion detection system based on continual learning," in *IEEE International Conference on Cyber Security and Resilience (CSR)*, 2023, pp. 86–91.

[11] Z. Song, Z. Zhao, F. Zhang, G. Xiong, G. Cheng, X. Zhao, S. Guo, and B. Chen, "I^2RNN: An Incremental and Interpretable Recurrent Neural Network for Encrypted Traffic Classification," *IEEE Trans. Depend. Sec. Comput.*, 2023.

[12] L. Du, Z. Gu, Y. Wang, L. Wang, and Y. Jia, "A few-shot class-incremental learning method for network intrusion detection," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 2, pp. 2389–2401, 2023.

[13] Y. Wang and S. Cao, "A Two-Stage Class Incremental Learning Approach for Network Intrusion Detection," in *IEEE Global Communications Conference (GLOBECOM)*, 2024, pp. 2353–2358.

[14] X. Xu, X. Zhang, Q. Zhang, Y. Wang, B. Adebisi, T. Ohtsuki, H. Sari, and G. Gui, "Advancing malware detection in network traffic with self-paced class incremental learning," *IEEE Internet Things J.*, 2024.

[15] F. Cerasuolo, G. Bovenzi, D. Ciuonzo, and A. Pescapè, "Attack-adaptive network intrusion detection systems for IoT networks through class incremental learning," *Computer Networks*, p. 111228, 2025.

[16] ——, "Adaptable, incremental, and explainable network intrusion detection systems for internet of things," *Engineering Applications of Artificial Intelligence*, vol. 144, p. 110143, 2025.

[17] V. Rey, P. M. S. Sánchez, A. H. Celdrán, and G. Bovet, "Federated learning for malware detection in IoT devices," *Computer Networks*, vol. 204, p. 108693, 2022.

[18] Z. Tang, H. Hu, and C. Xu, "A federated learning method for network intrusion detection," *Concurrency and Computation: Practice and Experience*, vol. 34, no. 10, p. e6812, 2022.

[19] O. Friha, M. A. Ferrag, L. Shu, L. Maglaras, K.-K. R. Choo, and M. Nafaa, "FELIDS: Federated learning-based intrusion detection system for agricultural Internet of Things," *Journal of Parallel and Distributed Computing*, vol. 165, pp. 17–31, 2022.

[20] J. Li, X. Tong, J. Liu, and L. Cheng, "An efficient federated learning system for network intrusion detection," *IEEE Syst. J.*, vol. 17, no. 2, pp. 2455–2464, 2023.

[21] M. J. Idrissi, H. Alami, A. El Mahdaouy, A. El Mekki, S. Oualil, Z. Yartaoui, and I. Berrada, "Fed-anids: Federated learning for anomaly-based network intrusion detection systems," *Expert Systems with Applications*, vol. 234, p. 121000, 2023.

[22] M. Bhavsar, Y. Bekele, K. Roy, J. Kelly, and D. Limbrick, "FL-IDS: Federated learning-based intrusion detection system using edge devices for transportation IoT," *IEEE Access*, 2024.

[23] M.-y. Zhu, Z. Chen, K.-f. Chen, N. Lv, and Y. Zhong, "Attention-based federated incremental learning for traffic classification in the Internet of Things," *Computer Communications*, vol. 185, pp. 168–175, 2022.

[24] R. R. dos Santos, E. K. Viegas, A. O. Santin, and P. Tedeschi, "Federated learning for reliable model updates in network-based intrusion detection," *Computers & Security*, vol. 133, p. 103413, 2023.

[25] D. Jin, S. Chen, H. He, X. Jiang, S. Cheng, and J. Yang, "Federated incremental learning based evolvable intrusion detection system for zero-day attacks," *IEEE Netw.*, vol. 37, no. 1, pp. 125–132, 2023.

[26] Z. Jin, J. Zhou, B. Li, X. Wu, and C. Duan, "FL-IIDS: A novel federated learning-based incremental intrusion detection system," *Future Generation Computer Systems*, vol. 151, pp. 57–70, 2024.

[27] J. Mao, Z. Wei, B. Li, R. Zhang, and L. Song, "Towards Ever-Evolution Network Threats: A Hierarchical Federated Class-Incremental Learning Approach for Network Intrusion Detection in IIoT," *IEEE Internet Things J.*, 2024.

[28] Z. Zhang, Y. Zhang, H. Li, S. Liu, W. Chen, Z. Zhang, and L. Tang, "Federated continual representation learning for evolutionary distributed intrusion detection in Industrial Internet of Things," *Engineering Applications of Artificial Intelligence*, vol. 135, p. 108826, 2024.

[29] N. H. Quyen, N. V. Hoang, P. T. Duy, and V.-H. Pham, "FI-IDS: A Federated Incremental Learning Approach for Intrusion Detection System," in *IEEE International Conference on Advanced Technologies for Communications (ATC)*, 2024, pp. 432–437.

[30] H. A. Tran, H. T. T. Binh, and A. Mellouk, "Federated learning for network traffic classification: A knowledge consolidation approach," *IEEE Trans. Netw. Sci. Eng.*, 2025.

[31] M. Masana, X. Liu, B. Twardowski, M. Menta, A. D. Bagdanov, and J. Van De Weijer, "Class-incremental learning: survey and performance evaluation on image classification," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 5, pp. 5513–5533, 2022.

[32] Q. Li, Z. Wen, Z. Wu, S. Hu, N. Wang, Y. Li, X. Liu, and B. He, "A survey on federated learning systems: Vision, hype and reality for data privacy and protection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 3347–3366, 2021.

[33] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, "Advances and open problems in federated learning," *Foundations and trends® in machine learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[34] G. Bovenzi, A. Nascita, L. Yang, A. Finamore, G. Aceto, D. Ciuonzo, A. Pescapé, and D. Rossi, "Benchmarking class incremental learning in deep learning traffic classification," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 1, pp. 51–69, 2023.

[35] G. Bovenzi, L. Yang, A. Finamore, G. Aceto, D. Ciuonzo, A. Pescapè, and D. Rossi, "A first look at class incremental learning in deep learning mobile traffic classification," in *Proc. IFIP Traffic Monitoring Analysis (TMA)*, 2021.

[36] Y. Wu, Y. Chen, L. Wang, Y. Ye, Z. Liu, Y. Guo, and Y. Fu, "Large scale incremental learning," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 374–382.

[37] R. Carillo, F. Cerasuolo, G. Bovenzi, D. Ciuonzo, and Pescapè, "Explainable Federated Class Incremental Learning for Encrypted Network Traffic Classification," *Computer Networks*, vol. 245, p. 110374, 2025.

[38] F. Cerasuolo, A. Nascita, G. Bovenzi, G. Aceto, D. Ciuonzo, A. Pescapè, and D. Rossi, "MEMENTO: A novel approach for class incremental learning of encrypted traffic," *Computer Networks*, vol. 245, p. 110374, 2024.

[39] D.-W. Zhou, Q.-W. Wang, Z.-H. Qi, H.-J. Ye, D.-C. Zhan, and Z. Liu, "Class-incremental learning: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2024.

[40] J. Zhang, F. Li, F. Ye, and H. Wu, "Autonomous unknown-application filtering and labeling for dl-based traffic classifier update," in *IEEE Conference on Computer Communications (INFOCOM)*, 2020, pp. 397–405.

[41] H. Chen, S. Huang, D. Zhang, M. Xiao, M. Skoglund, and H. V. Poor, "Federated learning over wireless iot networks with optimized communication and resources," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 16 592–16 605, 2022.

[42] X. Gao, X. Yang, H. Yu, Y. Kang, and T. Li, "Fedprok: Trustworthy federated class-incremental learning via prototypical feature knowledge transfer," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 4205–4214.

[43] J. Dong, H. Li, Y. Cong, G. Sun, Y. Zhang, and L. Van Gool, "No one left behind: Real-world federated class-incremental learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2023.

[44] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.

[45] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proceedings of Machine learning and systems*, vol. 2, pp. 429–450, 2020.

[46] D. A. E. Acar, Y. Zhao, R. M. Navarro, M. Mattina, P. N. Whatmough, and V. Saligrama, "Federated learning based on dynamic regularization," *arXiv preprint arXiv:2111.04263*, 2021.

[47] T. M. Booij, I. Chiscop, E. Meeuwissen, N. Moustafa, and F. T.

den Hartog, "Ton_IoT: The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion datasets," *IEEE Internet Things J.*, 2021.

[48] H. Kang, D. H. Ahn, G. M. Lee, J. D. Yoo, K. H. Park, and H. K. Kim, "IoT network intrusion dataset," 2019. [Online]. Available: https://dx.doi.org/10.21227/q70p-q449

[49] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022.

[50] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapé, "Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges," *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 2, pp. 445–458, 2019.

[51] W. John and S. Tafvelin, "Analysis of Internet backbone traffic and header anomalies observed," in *Proc. 7th ACM SIGCOMM Conference on Internet Measurement (IMC)*, 2007, pp. 111–116.

[52] L. Peng, B. Yang, and Y. Chen, "Effective packet number for early stage internet traffic identification," *Neurocomputing*, vol. 156, pp. 252–267, 2015.

[53] M. Wang, N. Yang, and N. Weng, "Exploring the Impact of Early Detection on DL-Based NIDSs Models," in *IEEE 14th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, 10 2023, pp. 0684–0691.

[54] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Network traffic classifier with convolutional and recurrent neural networks for Internet of Things," *IEEE Access*, vol. 5, pp. 18 042–18 050, 2017.